
WebChallenger: A Reliable and Efficient Generalist Web Agent

Jayoo Hwang*
ML Collective
jayoohm350@gmail.com

Xiaowen Zhang
longsurf.ai
sean@longsurf.ai

Vedant Padwal
Independent
vedantpadwalinfi@gmail.com

Abstract

Autonomous web navigation remains challenging for LLM agents, and the strongest generalist systems rely on proprietary reasoning models whose inference cost is prohibitive for the repetitive tasks where such agents would be most useful. We argue this gap stems not from insufficient model capability but from agent architectures that fail to replicate three human cognitive advantages: selective attention to relevant page regions, persistent memory of website structure, and procedural fluency with common interaction patterns. We introduce WebChallenger, a web agent framework that addresses each gap through architecture design rather than model scale, built around PageMem: a structured page representation deterministically constructed from the DOM that exposes each page as a hierarchy of semantic sections with short summaries. On this shared substrate we build three mechanisms that mirror the three cognitive advantages: a divide-and-conquer observation pipeline that lets the agent skim section summaries and extract details only from task-relevant regions; a lightweight exploration and memory system that traverses each website once to build a reusable map of pages and element behaviors; and compound action workflows that collapse common multi-step interactions into single agent actions, handling partial state changes automatically. Because all three operate over PageMem, the framework generalizes across websites without site-specific adapters. Using off-the-shelf open-weight models without fine-tuning, our system achieves 56.3% on WebArena, 48.7% on VisualWebArena, 51.0% on Online-Mind2Web, and 70.9% on WorkArena, approaching frontier proprietary systems at a fraction of the cost. Our code is released at this URL.

1 Introduction

“I touch the future. I teach”
— Christa McAuliffe

Autonomous web navigation has long been a goal of AI research (Doorenbos et al., 1997): the web is one of the most complex interactive environments available, and navigating it autonomously has broad practical implications, from automating repetitive knowledge work to serving as a testbed for general-purpose agent capabilities. Recent advances in large language models and vision-language models have driven rapid progress on computer-using agents (Marino & Marasović, 2025), yet even the strongest LLM agents remain below human performance on realistic, long-horizon web tasks (Jang et al., 2026; Miyai et al., 2025). Additionally, the best generalist agents rely on proprietary reasoning models whose inference cost is prohibitive for the repetitive work where agents would be desirable.

*Corresponding author.

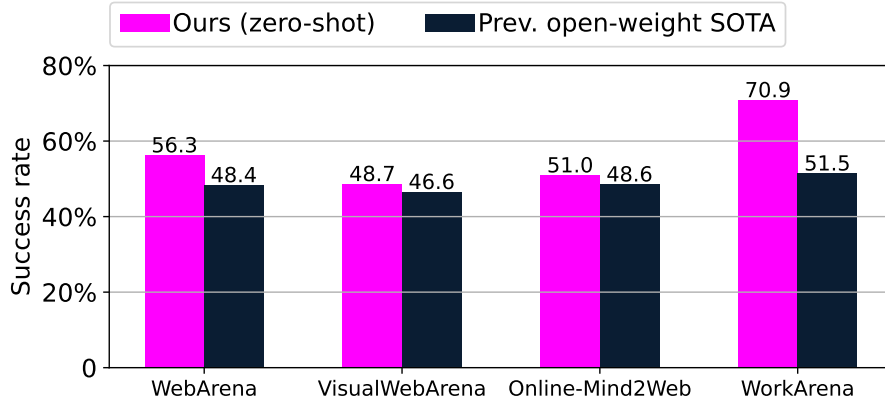


Figure 1: Benchmark results. WebChallenger sets new state-of-the-art performance among agents using open models across four web navigation benchmarks. Our results were obtained with far less compute than the baselines which either used finetuning or larger models, demonstrating that **scaffolding alone can drastically improve web agent performance**.

This gap echoes Moravec’s paradox (Moravec, 1988; Su, 2025): browsing the web is effortless for humans yet remarkably difficult for AI models that excel at mathematics and code generation. We argue that this difficulty stems not from a lack of web knowledge in current models, but from a mismatch between how agent frameworks present the web environment and how it needs to be processed. Specifically, humans bring three cognitive advantages to web navigation that current agent architectures fail to replicate. First, *selective attention*: humans focus on relevant regions of a page while ignoring the rest (Putkonen et al., 2023), whereas LLM agents ingest entire pages as flat token sequences, diluting relevant information in irrelevant context. Second, *persistent memory*: humans memorize the layout and functionality of websites they have used before, while LLM agents approach each session with no prior environmental knowledge. Third, *procedural fluency*: humans internalize reusable routines for common interaction patterns (e.g., searching, selecting from a dropdown, filling a form) that execute as cohesive sequences without deliberate reasoning at each step, while LLM agents must re-observe and re-reason over the full page state for every atomic action.

In this work, we show that these three human advantages can be realized through agent architecture design rather than model scale or training. Implementing them in a way that generalizes across websites without site-specific adapters requires a shared abstraction the agent can reason over uniformly. We introduce **PageMem**, a structured page representation deterministically constructed from the DOM that exposes each page as a hierarchy of semantic sections with short summaries: a representation the agent can skim like a table of contents, expand selectively for detail, and dispatch to specialized workflows by section type. On this substrate we build three mechanisms that mirror the three cognitive advantages above.

A **divide-and-conquer observation** pipeline lets the agent skim PageMem’s section summaries, select task-relevant regions, and extract details only from those regions, producing information-dense observations without processing entire pages.

A lightweight **exploration and memory** system traverses new websites before task execution, assembling a persistent collection of PageMems that records pages, navigation paths, and interactive element behaviors.

Compound action workflows implement site-agnostic routines for common interaction patterns such as searching, menu selection, and form submission. Dispatched by section type, these workflows collapse multi-step processes into single agent actions and automatically surface partial state changes (such as a dropdown expanding) without requiring the agent to reprocess the full page.

Decomposing observation and decision-making into focused sub-prompts in this way allows our framework to extract strong performance from small, locally-run models that would struggle with the monolithic prompts used by most existing agent frameworks. Using an off-the-shelf 32B LLM and a 7B VLM without any fine-tuning, our system achieves 56.3% on WebArena (Zhou et al., 2024),

48.7% on VisualWebArena (Koh et al., 2024), 51.0% on Online-Mind2Web (Xue et al., 2025), and 70.9% on WorkArena (Drouin et al., 2024) — state-of-the-art results among open-weight models of comparable scale, and approaching frontier proprietary systems at a fraction of the inference cost. These results indicate that current LLMs already possess sufficient reasoning ability for many web tasks; what they lack is the right scaffolding around observation, memory, and action to use it effectively.

2 Method

2.1 Problem Formulation

We frame web navigation as a sequential decision process in which an agent interacts with a web browser to complete a natural-language task. A task is a tuple $\tau = (I, u_0)$ consisting of an instruction I and a starting URL u_0 , which determines the initial website w_0 from a set $\mathcal{W}$ of target websites. At each timestep t , the agent receives an observation o_t , maintains a compact history h_t of prior interactions, and selects an action a_t from a candidate set $\mathcal{A}_t$.

A standard LLM web agent implements this loop as $a_t = \pi(o_t, h_t)$: a single model call that maps a raw observation — typically a full accessibility tree or screenshot — and an interaction history to the next atomic browser action. Our system departs from this template with four novel components.

A structured page representation. Rather than exposing the raw DOM or accessibility tree, we introduce **PageMem**, a structured representation p deterministically constructed from the DOM. Each PageMem contains an ordered list of **PageSections** $\{s_1, \dots, s_n\}$ corresponding to semantic regions of the page, and each PageSection contains a set of interactive **Elements**. PageSections carry model-generated summaries alongside DOM-derived attributes, and serve as the shared substrate on which the observation pipeline, memory, and action workflows all operate. This abstract substrate is what allows the rest of the system to remain site-agnostic. PageMem is defined in detail in §2.2.

Persistent memory from offline exploration. Before any task is attempted, an offline exploration phase traverses each website $w \in \mathcal{W}$ and builds a **WebsiteMem** $\mathcal{M}_w$: a persistent collection of PageMems indexed by URL, together with information about page templates and element behaviors discovered during exploration. At task start the agent may select a set of bookmarks $B_\tau \subseteq \mathcal{M}_{w_0}$ that remain available as navigation targets throughout the task. WebsiteMem is constructed once per site and reused across all subsequent tasks. Exploration and memory are detailed in §2.3.

A multi-stage observation pipeline. Rather than producing o_t by serializing the full page, we decompose observation into three stages over the current PageMem p_t : the agent first selects a subset of sections whose summaries appear relevant to the task, then extracts task-relevant details from the full content of each selected section, and finally synthesizes the extractions into a task-focused page summary $\hat{o}_t$. The pipeline is defined in §2.4.

Compound actions with workflows. A timestep in our system corresponds to one *high-level* agent action, which may execute multiple browser operations. Single-step actions (clicking a link, navigating to a URL) cause a page transition and advance the loop directly. Compound actions (dropdown selection, form submission, search) invoke a **workflow** $\omega(a_t)$ — a fixed sequence of additional LLM sub-calls and browser operations that handles intermediate partial state changes, such as a dropdown expanding or form fields being filled one at a time, before returning control to the top-level loop. The action system is detailed in §2.5.

2.1.1 System overview.

Given a task $\tau = (I, u_0)$, the agent retrieves the WebsiteMem $\mathcal{M}_{w_0}$ built during offline exploration and optionally selects bookmarks B_τ . At each timestep t , it (i) retrieves or constructs the PageMem p_t for the current page; (ii) applies the observation pipeline to produce $\hat{o}_t$; and (iii) selects an action $a_t \in \mathcal{A}_t$, which executes either as a direct browser operation or through a workflow $\omega(a_t)$. The loop terminates when the agent selects an end-task action and verifies completion, or when a step budget is exhausted. The agent inference algorithm is provided in Appendix A.4.

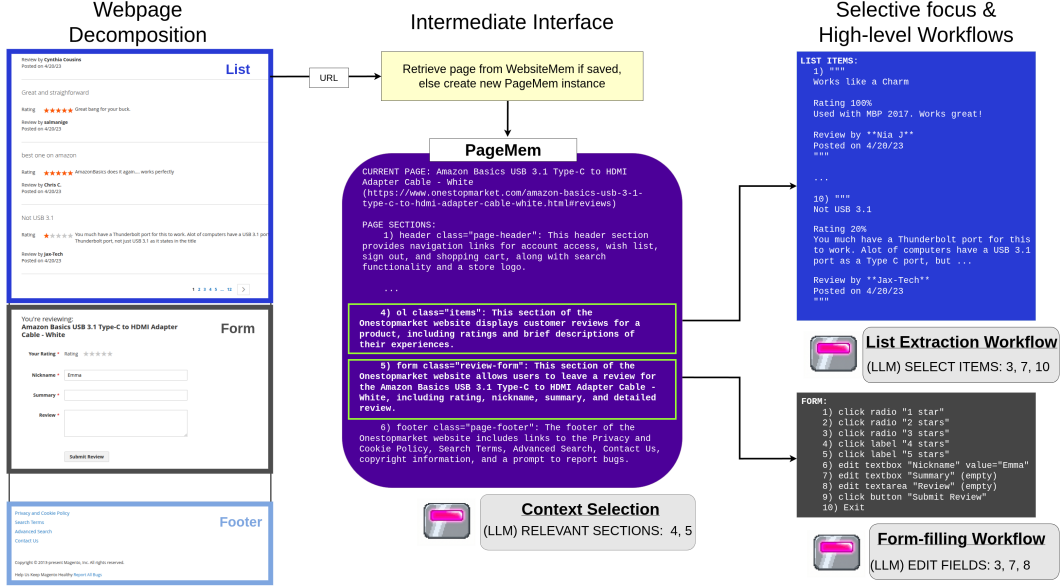


Figure 2: Overview of WebChallenger. (left) Each webpage is decomposed along the DOM into sections that correspond to semantic regions of the page. (middle) These sections are indexed by short summaries to form a PageMem, a structured page representation cached in per-website memory. The agent skims these summaries and expands only the task-relevant sections for detailed processing. (right) Specialized multi-step workflows are executed based on section type.

2.2 PageMem

PageMem is an abstract page representation deterministically constructed from the DOM that serves as the common interface shared by the exploration (§2.3), observation (§2.4), and action (§2.5) components. It exposes a semantic, chunked view of a page while preserving the selectors needed for direct browser control, allowing higher-level components to operate on abstract objects without site-specific adapters.

Hierarchy. PageMem is organized in four levels. A *WebsiteMem* $\mathcal{M}_w$ contains all PageMems and elements encountered on a website w . A *PageMem* p corresponds to a single page and holds a title, URL, ordered list of sections ($s_1, \dots, s_n$), and a page-level summary. A *PageSection* s_i represents a subregion of the page (e.g., navigation bar, product listing, review form) and maps to a sub-tree of the DOM. Each section carries DOM-derived state attributes (e.g., tag, class, bounding box, contained elements) and variable metadata (e.g., summary, extracted details). An *Element* e represents a single interactive widget, and carries DOM attributes to enable selector construction as well as metadata such as the element’s current value, clicked status, and dropdown elements. The PageMem data structure acts as the central hub where all agent-related information about a page is stored, flexibly facilitating the implementation of precise context-engineering for web agents.

Construction. PageSections are produced by recursively splitting the DOM tree, terminating at nodes that either fall below a size threshold or match a grouping tag (form, ul, li, table, section, etc.); sibling nodes sharing tag and class are grouped into a single *list section*. Clickable elements are identified using heuristics adapted from the BrowserUse library (Müller & Žunic., 2024) and assigned to their ancestor section. Finally, we prompt an LLM or VLM to provide a general one-sentence summary for each section and the overall page. Normal sections are size-bounded so their full content fits in a single LLM call; list sections are unbounded and represented at a higher level of abstraction as a sequence of uniform sub-sections, one per item. Full details and construction algorithm are given in Appendix A.1.

2.3 Exploration and Memory

Before any task is attempted, an offline exploration phase traverses each target website $w \in \mathcal{W}$ and produces the WebsiteMem $\mathcal{M}_w$ used at inference. Exploration is fully deterministic: it requires

no LLM guidance, task demonstrations, or external resources. Compared to tree-search methods that expand during execution or skill-learning approaches that improve only after accumulating task experience, our approach amortizes environmental knowledge upfront and makes it available from the first task at a fixed, one-time cost. We describe exploration here and provide details in Appendix A.2.

Traversal. Starting from the homepage of a website, we explore all unique clickable elements on the page in order. If a page contains many repeated elements with the same structure (such as a list or table of results), then we only explore elements contained within one item/row of the list/table for efficiency. We skip exploring elements that have already been explored on the current website. An element is explored by clicking it and recording the state-transition it induces. If clicking results in navigation to an *unexplored page*, the URL of the new page is added to the exploration frontier. If clicking an element modifies the state of the *current page* by expanding an interface, then we add the newly revealed elements as the clicked element’s dropdown items. After exploring the homepage, we repeat the above process for the pages that were added to the exploration frontier. For each page visited, we extract its title and summarize it. Exploration continues depth-first until a set maximum search depth is reached. We also limit the maximum number of elements explored per page, the total number of pages explored, and also set a timeout for each website.

Use at inference. $\mathcal{M}_w$ is saved per-site as JSON and reused across tasks. This memory is consumed by the agent in a highly token-efficient manner: rather than loading the full memory into the context window or retrieving large passages of text, only a handful of extra tokens are added per prompt. At task start the agent may select a small bookmark set $B_\tau \subseteq \mathcal{M}_w$ that remains available as navigation shortcuts throughout the task, and the agent’s observation space is augmented to provide context about hidden dropdown menu elements.

We adopt a deliberately minimal memory instantiation in this work in order to efficiently demonstrate how our framework can be used to create structured site-specific memories for LLM agents. Our website representation could also be used as a building block to implement more advanced memory approaches such as those explored in Agent Workflow Memory (Wang et al., 2024) or SkillWeaver (Zheng et al., 2025a). We leave this to future work.

2.4 Divide-and-Conquer Observation

Large pages can easily exceed the reliable context window of small LLMs, and even within that window, flattening a full accessibility tree into a single prompt dilutes task-relevant signal among boilerplate. Our system addresses these issues by decomposing page analysis across multiple focused sub-prompts, extracting and condensing task-relevant information into a summarized observation $\hat{o}_t$.

PageMem retrieval and update. For the current page p_t , the agent first checks whether a PageMem already exists in $\mathcal{M}_w$ (either built during exploration or cached from a previous visit in the same task). If so, it is reused; otherwise a fresh PageMem is constructed from the live DOM as described in §2.2. When a cached PageMem is reused, sections whose elements have changed since the last visit are re-summarized while unchanged sections retain their cached summaries and extractions, amortizing summarization cost across timesteps and repeat visits.

Section selection. The LLM is shown the list of section summaries for p_t along with the task instruction I and the interaction history h_t , and returns a subset of sections $S_t \subseteq \{s_1, \dots, s_n\}$ judged relevant to the task.

Detail extraction. For each PageSection $s \in S_t$, the LLM is prompted to extract task-relevant information from the section’s full content (accessibility sub-tree, page metadata). If a section contains visible images above a minimum size, the URLs and VLM descriptions for the images are included in the extraction prompt. When s is a table or list section, extraction is preceded by an item-selection step: items are grouped into chunks of maximum size c , the LLM selects relevant items from each chunk, and only the selected items are passed to the extraction prompt (Figure 2, middle-right). This chunked filtering keeps even very long lists or tables within context. Extractions are cached on the section and reused while the section is unchanged. Full process is in Appendix A.3.

Summary synthesis. Finally, we provide the LLM with the extracted outputs from all selected sections and prompt it to generate a compact page summary $\hat{o}_t$, which becomes the observation passed to the action module and appended to the history h_t . We instruct the LLM to generate a one paragraph long summary, as we find this is sufficient in most cases to capture the task-relevant page information while also allowing the history representation to remain compact.

2.5 Compound Actions and Workflows

At each timestep t the action module selects an action a_t from a candidate set $\mathcal{A}_t$ assembled from the current PageMem, the selected sections S_t , and the agent’s memory. Rather than exposing actions through an LLM tool-use interface, we present $\mathcal{A}_t$ as a numbered list and prompt the model to return the index of its chosen action, as we find this to be more reliable than tool use for small open-weight models. Our system then automatically executes the appropriate action function based on the action selected. Many actions are *compound*: their execution invokes a workflow $\omega(a_t)$ that combines multiple LLM sub-calls with browser operations (implemented using Playwright²) to complete a multi-step interaction as a single action.

Action selection. $\mathcal{A}_t$ comprises three groups. *Navigation* actions include previously visited URLs, bookmarks in B_τ , a type-URL action, and (when applicable) switch-tab and switch-website. *Element* actions are gathered from the selected sections S_t and a rule-based pre-filter removes no-ops and rarely-useful actions (e.g., navigating to the current page, clicking a selected radio). The LLM filters the full list of navigation and element actions for those it deems most promising for the task. *End task* is always available. The LLM then selects the next action a_t from the filtered candidate set.

Workflows. Our design principle for workflows is that action sequences whose intermediate steps produce only *partial* state changes to the current page (dropdown expansion, search suggestions, field-by-field form entry) are collapsed into a single compound action handled by a workflow, while actions that navigate to a different page are kept as individual agent actions. This keeps the agent’s decision loop anchored at semantically meaningful transitions rather than at every micro-interaction. We describe two representative workflows; the full set is summarized in Appendix A.5.

Dropdown selection. The workflow clicks the dropdown element, extracts the list of revealed options via a section-level diff, prompts the LLM to choose one option by index, and clicks the chosen option.

Form submission. The LLM first selects which fields of the form to fill, is then prompted for each field’s value (which is entered on the page), and finally reviews the completed form to either edit further or submit (Figure 2, bottom-right). The workflow handles field-specific details internally so that the agent-level action is a single SubmitForm step.

End-task. The end-task action invokes a one-time verification workflow: the LLM is prompted with the task and history and asked to either produce a final answer or report that the task is not yet complete, in which case the LLM is re-prompted to select a different non-terminating action.

3 Experiments

We evaluate WebChallenger on four open-ended web navigation benchmarks to test its performance on a diverse range of capabilities. **WebArena** (Zhou et al., 2024) consists of 812 tasks in 6 simulated environments that are designed to mimic common website types (e.g., forum, wiki) and uses a combination of both programmatic and LLM evaluation. **VisualWebArena** (Koh et al., 2024) builds on the infrastructure of WebArena, but consists of 910 tasks that require visual reasoning. **Online-Mind2Web** (Xue et al., 2025) consists of 300 tasks across 136 real-world websites. We score our agent using human evaluations for Online-Mind2Web. **WorkArena** (Drouin et al., 2024) contains 330 enterprise-related tasks that require agents to navigate complex user interfaces.

3.1 Experimental Setup

We use GLM-4-32B-0414 (Z.ai, 2025) as the LLM controller, and Qwen2.5-VL-7B-Instruct (Bai et al., 2025b) as our supplementary vision model for image captioning. For VisualWebArena, we

²<https://playwright.dev/python/>

Table 1: Benchmark success rates (%). WebChallenger sets new open-model SOTA on four web navigation benchmarks and performs comparably to agents built on proprietary models, despite using no training. Best proprietary and open-model results are bolded. VWA: VisualWebArena (Koh et al., 2024), O-M2W: Online-Mind2Web (Xue et al., 2025), WoA: WorkArena (Drouin et al., 2024).

	Method	Model(s)	WebArena	VWA	O-M2W	WoA L1
Proprietary Models	GenericAgent	GPT-4o	31.4	26.7	-	45.5
	GenericAgent	Claude 3.5 Sonnet	36.2	21.0	-	56.4
	GenericAgent	GPT-5	-	-	-	79.1
	GenericAgent	GPT-4o-mini	17.4	16.9	-	27.0
	WALT	GPT-5	50.1	52.9	-	-
	IBM CUGA	-	61.7	-	-	-
	OpenAI CUA	-	58.1	-	61.3	-
	ScribeAgent	GPT-4o + Qwen2.5-32B	53.0	-	-	-
	AgentSymbiotic	Claude 3.5 + Llama3 8B	48.5	-	-	-
	AgentOccam-Judge	GPT-4	45.7	-	-	-
	WebPilot	GPT-4o	37.2	-	-	-
	Agent Workflow Memory	GPT-4o	35.5	-	-	-
	SkillWeaver	GPT-4o	29.8	-	-	-
Open-Source Models (fine-tuned)	Agent-as-Annotators	Qwen3.5-9B	41.5	33.9	-	51.5
	Mobile-Agent-v3.5	Qwen3-VL-32B	48.4	46.6	-	-
	WebDreamer	Qwen2-VL-7B	-	21.9	35.0	-
	Fara-7B	Qwen2.5-VL-7B	-	-	34.1	-
	Learn-by-Interact	Codestral 22B	24.2	-	-	-
	AgentTrek	Qwen2.5-32B	22.4	-	-	-
	Go-Browse	Qwen2.5-7B	21.7	-	-	-
	AutoWebGLM	ChatGLM3 6B	18.2	-	-	-
	TTI	Gemma 3 12B	26.1	-	-	-
	GenericAgent	GPT-oss-120b	-	-	-	50.9
(zero-shot)	Tree Search	Llama-3-70B-Instruct	10.1	16.7	-	-
	WebChallenger (Ours)	GLM-4-32B + Qwen2.5-VL-7B	56.3	48.7[†]	51.0	70.9

[†] Our VisualWebArena experiments use Qwen3-VL-4B-Instruct in place of Qwen2.5-VL-7B-Instruct.

use Qwen3-VL-4B-Instruct (Bai et al., 2025a) as the vision model. For all experiments we use the same agent prompts and sample with temperature 0. For each benchmark, we first explore the full set of benchmark websites before running inference. During inference, the agent’s memory is reset between tasks to the post-exploration state to preserve independence between evaluation samples. Additional experiment details are provided in Appendix B.

3.2 Main Results

Baselines. We compare WebChallenger against strong open model and proprietary baselines for each of our selected benchmarks.

For proprietary model baselines, we use WALT (Prabhu et al., 2025), IBM CUGA (Marreed et al., 2025), OpenAI CUA (OpenAI, 2025), ScribeAgent (Shen et al., 2024), AgentSymbiotic, (Zhang et al., 2025b), AgentOccam-Judge (Yang et al., 2025), WebPilot (Zhang et al., 2024), SkillWeaver (Zheng et al., 2025a), and Agent Workflow Memory (Wang et al., 2024).

For open-model baselines, we use Agent-as-Annotators (Lù & Reddy, 2026), Mobile-Agent-v3.5 (Xu et al., 2026), WebDreamer (Gu et al., 2025), Fara-7B (Awadallah et al., 2025), Learn-by-Interact (Su et al., 2025), AgentTrek (Xu et al., 2025), Go-Browse (Gandhi & Neubig, 2025), AutoWebGLM (Lai et al., 2024), TTI (Shen et al., 2025), and Tree Search (Koh et al., 2025).

GenericAgent results are taken from the official BrowserGym leaderboard (ServiceNow, 2025). All other baseline results in Table 1 are taken from their original reports.

Results. As shown in Table 1, WebChallenger sets new state-of-the-art results among open-model agents on all four benchmarks despite using no fine-tuning. On WebArena, our 56.3% exceeds the

Table 2: Component ablations on WebArena-lite (165 tasks). Per-site task counts: Shopping ($n = 46$), Reddit ($n = 21$), GitLab ($n = 32$), Maps ($n = 31$), CMS ($n = 35$). Δ is the change in average success rate relative to the full system.

Method	Shopping	Reddit	GitLab	Maps	CMS	Avg	Δ
<i>WebChallenger</i> (GLM-4-32B)	56.5	71.4	65.5	45.1	60.0	58.8	–
– memory	43.4	71.4	53.1	48.4	54.2	51.2	–7.6
– compound actions	47.8	66.7	50.0	48.4	40.0	49.1	–9.7
– observation pipeline	48.9	57.1	53.1	26.7	27.3	41.2	–17.6

Table 3: Token and step usage for the GLM-4-32B component ablations. Tokens/Prompt is the average input token count per LLM call. We count compound actions as one step.

Method	Avg Steps	Total Tokens	Tokens/Prompt
<i>WebChallenger</i> (GLM-4-32B)	7.20	47.0M	1850
– memory	7.34	48.1M	1781
– compound actions	9.85	64.9M	2109
– observation pipeline	11.26	36.0M	8793

strongest fine-tuned open-model baseline (Mobile-Agent-v3.5, 48.4%) by 7.9 points and surpasses ScribeAgent (53.0%, GPT-4o planner). On VisualWebArena, 48.7% outperforms all open-model baselines and trails only WALT (52.9%, GPT-5). On WorkArena, 70.9% lands 20 points above the next-best zero-shot open model and exceeds both Claude 3.5 Sonnet (56.4%) and GPT-4o (45.5%) backbones. 51.0% on Online-Mind2Web shows that our framework generalizes by exploiting structural patterns shared across the web rather than site-specific adaptations. These results demonstrate that careful architectural scaffolding can close most of the gap between small open-weight models and frontier proprietary systems on long-horizon web tasks, and that a single configuration generalizes consistently across a wide range of tasks and environments.

3.3 Analysis

We run additional experiments on the 165-task WebArena-lite subset (Liu et al., 2024) to examine component contributions, compute usage, and backbone sensitivity; our system’s lite score (58.8) tracks the full WebArena score (56.3) closely.

Component ablations. We separately remove each of the three architectural components (Table 2). *Remove memory* disables bookmarks, dropdown information, and pre-cached section summaries; PageMem is still used at inference time but constructed from scratch for each task. *Remove compound actions* restricts the agent to single basic actions (CLICKELEMENT, ENTERINPUT, SELECTOPTION, UPLOADFILE, plus navigation), eliminating the search, dropdown, and form-filling workflows. *Remove observation pipeline* replaces section selection and detail extraction with a single prompt containing the full ax-tree and all available actions, with history reduced to a list of prior actions.

Among the three components, removing the observation pipeline causes the largest accuracy drop (–17.6 points), followed by compound actions (–9.7) and memory (–7.6). Compound action removal has its largest effect on CMS (–20.0), as CMS involves interactions with complex interfaces such as forms and filtering menus. On Reddit, removing memory has no effect (71.4 in both conditions), suggesting GLM-4-32B navigates Reddit reliably without pre-cached information. Maps performance is largely unaffected by memory and compound actions, as the Maps environment is focused on a single interface that doesn’t benefit from those components.

Token and step efficiency. Removing the observation pipeline reduces total tokens (47.0M $\rightarrow$ 36.0M) but raises average prompt size $4.75\times$ (1850 $\rightarrow$ 8793 tokens) and step count from 7.2 to 11.26 (Table 3). Our multi-stage observation processing decomposes large difficult prompts into several smaller but easier prompts, trading inference compute for performance. Compound actions significantly improve agent efficiency: removal causes total tokens to rise to 64.9M and steps to 9.85, since interactions that previously executed within a single workflow now require a separate observation and decision cycle per atomic action.

Table 4: Backbone model comparison on WebArena-lite (Liu et al., 2024). The bottom row uses the minimal GenericAgent harness from BrowserGym (Chezelles et al., 2025) with the same GLM-4-32B model used in our system to isolate the contribution of our harness.

Method	Shopping	Reddit	GitLab	Maps	CMS	Avg
<i>WebChallenger</i> w/ GPT-5	60.9	71.4	70.8	74.2	70.6	68.7
<i>WebChallenger</i> w/ GLM-4-32B	56.5	71.4	65.5	45.1	60.0	58.8
<i>WebChallenger</i> w/ GPT-4o-mini	36.9	57.1	43.7	51.6	51.4	46.7
GenericAgent w/ GLM-4-32B	19.5	23.8	28.1	16.1	11.4	19.4

Backbone model comparison. We swap the GLM-4-32B backbone for GPT-5 and GPT-4o-mini, and additionally evaluate GLM-4-32B alone in the minimal GenericAgent harness to isolate the architecture’s contribution (Table 4). GPT-5 reaches 68.7%, 9.9 points above GLM-4-32B in the same framework. GPT-4o-mini reaches 46.7%, indicating our framework retains strong performance even with weaker backbones. GLM-4-32B in the GenericAgent harness scores 19.4%, against 58.8% in our framework, a 39.4-point improvement from system architecture alone.

4 Related Work

Agent Memory. A growing body of work equips LLM web agents with external memory by accumulating insights from task trajectories (Wang et al., 2024; Ouyang et al., 2025; Sarch et al., 2025a; Pang et al., 2025; Liu et al., 2025; Nekoei et al., 2025; Fu et al., 2024; Chen et al., 2024; Cheng et al., 2025; Su et al., 2025). Our memory takes a complementary route: a deterministic exploration procedure efficiently produces a structured site map with no task experience, demonstrations, or documentation required, making it applicable to any website out of the box.

Web Action Space. Several works extend web agent action spaces beyond click and type by introducing higher-level programmatic skills (Song et al., 2025; Wang et al., 2025; Zheng et al., 2025a; He et al., 2025; Prabhu et al., 2025; Yu et al., 2025; Wang et al., 2026; Zhong et al., 2026). These approaches typically learn site-specific code, whereas our compound workflows operate over PageMem’s abstract elements and sections and generalize across sites with no per-site adaptation. We also depart from the standard tool-calling interface in favor of a numbered-list action format.

Observation Refinement. Web agents observe their environment through text (Gur et al., 2018; Li et al., 2023; Kim et al., 2023), screenshots (Shaw et al., 2023; Hong et al., 2024; Gou et al., 2025; Pahuja et al., 2025; He et al., 2024; Zheng et al., 2024; Verma et al., 2024), or both (Furuta et al., 2024). All such modalities are token-heavy and information-sparse, motivating refinement strategies. Text-based agents prune irrelevant HTML elements (Gur et al., 2024; Deng et al., 2023; Kil et al., 2024; Lù et al., 2024; Lee et al., 2025; Abuelsaad et al., 2024; Kerboua et al., 2025), while vision-based agents focus attention on specific screen regions (Sarch et al., 2025b; Singh et al., 2025; Luo et al., 2025; Park et al., 2025). We apply region-based focus to a hybrid text-vision agent by splitting pages along DOM structure, which preserves the semantic grouping authored into the page better than pixel-space cropping. Feuillade--Montixi (2026) explores a similar DOM-based approach. More broadly, our pipeline echoes a line of work on decomposing long-context tasks into focused sub-prompts (Zhang et al., 2026; Chen et al., 2023; Jayalath et al., 2025; Lee et al., 2024).

5 Conclusion

WebChallenger closes much of the gap between small open-weight models and frontier proprietary systems on long-horizon web navigation. We argue current LLMs already possess sufficient intelligence for many common web tasks, but standard frameworks fail to scaffold that intelligence with the selective attention, persistent memory, and procedural fluency humans rely on. We supply each through a divide-and-conquer observation pipeline, an offline exploration and memory system, and compound action workflows. These components are implemented on top of PageMem, a shared page representation that generalizes across websites without site-specific adapters. Using small, general-purpose models without fine-tuning, our system sets new state-of-the-art results among open-weight agents on four diverse web agent benchmarks.

Acknowledgements

We thank the ML Collective community for their support, discussions, and feedback.

References

- Abuelsaad, T., Akkil, D., Dey, P., Jagmohan, A., Vempaty, A., and Kokku, R. Agent-e: From autonomous web navigation to foundational design principles in agentic systems, 2024. URL <https://arxiv.org/abs/2407.13032>.
- Anthropic. Mitigating the risk of prompt injections in browser use. <https://www.anthropic.com/news/prompt-injection-defenses>, November 2025. Accessed: 2026-05-11.
- Awadallah, A., Lara, Y., Magazine, R., Mozannar, H., Nambi, A., Pandya, Y., Rajeswaran, A., Rosset, C., Taymanov, A., Vineet, V., Whitehead, S., and Zhao, A. Fara-7b: An efficient agentic model for computer use, 2025. URL <https://arxiv.org/abs/2511.19663>.
- Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., Ge, W., Guo, Z., Huang, Q., Huang, J., Huang, F., Hui, B., Jiang, S., Li, Z., Li, M., Li, M., Li, K., Lin, Z., Lin, J., Liu, X., Liu, J., Liu, C., Liu, Y., Liu, D., Liu, S., Lu, D., Luo, R., Lv, C., Men, R., Meng, L., Ren, X., Ren, X., Song, S., Sun, Y., Tang, J., Tu, J., Wan, J., Wang, P., Wang, P., Wang, Q., Wang, Y., Xie, T., Xu, Y., Xu, H., Xu, J., Yang, Z., Yang, M., Yang, J., Yang, A., Yu, B., Zhang, F., Zhang, H., Zhang, X., Zheng, B., Zhong, H., Zhou, J., Zhou, F., Zhou, J., Zhu, Y., and Zhu, K. Qwen3-vl technical report, 2025a. URL <https://arxiv.org/abs/2511.21631>.
- Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., Zhong, H., Zhu, Y., Yang, M., Li, Z., Wan, J., Wang, P., Ding, W., Fu, Z., Xu, Y., Ye, J., Zhang, X., Xie, T., Cheng, Z., Zhang, H., Yang, Z., Xu, H., and Lin, J. Qwen2.5-vl technical report, 2025b. URL <https://arxiv.org/abs/2502.13923>.
- Chen, H., Pasunuru, R., Weston, J., and Celikyilmaz, A. Walking down the memory maze: Beyond context limit through interactive reading, 2023. URL <https://arxiv.org/abs/2310.05029>.
- Chen, M., Li, Y., Yang, Y., Yu, S., Lin, B., and He, X. Automanual: Constructing instruction manuals by llm agents via interactive environmental learning, 2024. URL <https://arxiv.org/abs/2405.16247>.
- Cheng, J., Kumar, A., Lal, R., Rajasekaran, R., Ramezani, H., Khan, O. Z., Rokhlenko, O., Chiu-Webster, S., Hua, G., and Amiri, H. Webatlas: An llm agent with experience-driven memory and action simulation, 2025. URL <https://arxiv.org/abs/2510.22732>.
- Chezelles, T. L. S. D., Gasse, M., Drouin, A., Caccia, M., Boisvert, L., Thakkar, M., Marty, T., Assouel, R., Shayegan, S. O., Jang, L. K., Lù, X. H., Yoran, O., Kong, D., Xu, F. F., Reddy, S., Cappart, Q., Neubig, G., Salakhutdinov, R., Chapados, N., and Lacoste, A. The browsergym ecosystem for web agent research, 2025. URL <https://arxiv.org/abs/2412.05467>.
- Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., and Su, Y. Mind2web: Towards a generalist agent for the web, 2023. URL <https://arxiv.org/abs/2306.06070>.
- Doorenbos, R. B., Etzioni, O., and Weld, D. S. A scalable comparison-shopping agent for the world-wide web. In *Proceedings of the First International Conference on Autonomous Agents, AGENTS '97*, pp. 39–48, New York, NY, USA, 1997. Association for Computing Machinery. ISBN 0897918770. doi: 10.1145/267658.267666. URL <https://doi.org/10.1145/267658.267666>.
- Drouin, A., Gasse, M., Caccia, M., Laradji, I. H., Verme, M. D., Marty, T., Boisvert, L., Thakkar, M., Cappart, Q., Vazquez, D., Chapados, N., and Lacoste, A. Workarena: How capable are web agents at solving common knowledge work tasks?, 2024. URL <https://arxiv.org/abs/2403.07718>.
- Feuillade--Montixi, Q. WebFurl: A browser-use AI agent with compressed unfoldable HTML representation for high token efficiency. <https://github.com/WeaveMindAI/Webfurl>, 2026. GitHub repository.

- Fu, Y., Kim, D.-K., Kim, J., Sohn, S., Logeswaran, L., Bae, K., and Lee, H. Autoguide: Automated generation and selection of context-aware guidelines for large language model agents, 2024. URL <https://arxiv.org/abs/2403.08978>.
- Furuta, H., Lee, K.-H., Nachum, O., Matsuo, Y., Faust, A., Gu, S. S., and Gur, I. Multimodal web navigation with instruction-finetuned foundation models, 2024. URL <https://arxiv.org/abs/2305.11854>.
- Gandhi, A. and Neubig, G. Go-browse: Training web agents with structured exploration, 2025. URL <https://arxiv.org/abs/2506.03533>.
- Gou, B., Wang, R., Zheng, B., Xie, Y., Chang, C., Shu, Y., Sun, H., and Su, Y. Navigating the digital world as humans do: Universal visual grounding for gui agents, 2025. URL <https://arxiv.org/abs/2410.05243>.
- Gu, Y., Zhang, K., Ning, Y., Zheng, B., Gou, B., Xue, T., Chang, C., Srivastava, S., Xie, Y., Qi, P., Sun, H., and Su, Y. Is your llm secretly a world model of the internet? model-based planning for web agents, 2025. URL <https://arxiv.org/abs/2411.06559>.
- Gur, I., Rueckert, U., Faust, A., and Hakkani-Tur, D. Learning to navigate the web, 2018. URL <https://arxiv.org/abs/1812.09195>.
- Gur, I., Furuta, H., Huang, A., Safdari, M., Matsuo, Y., Eck, D., and Faust, A. A real-world webagent with planning, long context understanding, and program synthesis, 2024. URL <https://arxiv.org/abs/2307.12856>.
- He, H., Yao, W., Ma, K., Yu, W., Dai, Y., Zhang, H., Lan, Z., and Yu, D. Webvoyager: Building an end-to-end web agent with large multimodal models, 2024. URL <https://arxiv.org/abs/2401.13919>.
- He, K., Wang, Z., Zhuang, C., and Gu, J. Recon-act: A self-evolving multi-agent browser-use system via web reconnaissance, tool generation, and task execution, 2025. URL <https://arxiv.org/abs/2509.21072>.
- Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Zhang, Y., Li, J., Xu, B., Dong, Y., Ding, M., and Tang, J. Cogagent: A visual language model for gui agents, 2024. URL <https://arxiv.org/abs/2312.08914>.
- Jang, L. K., Koh, J. Y., Fried, D., and Salakhutdinov, R. Odysseys: Benchmarking web agents on realistic long horizon tasks, 2026. URL <https://arxiv.org/abs/2604.24964>.
- Jayalath, D., Wendt, J. B., Monath, N., Tata, S., and Gunel, B. Prism: Efficient long-range reasoning with short-context llms, 2025. URL <https://arxiv.org/abs/2412.18914>.
- Kerboua, I., Shayegan, S. O., Thakkar, M., Lù, X. H., Boisvert, L., Caccia, M., Espinas, J., Aussem, A., Eglin, V., and Lacoste, A. Focusagent: Simple yet effective ways of trimming the large context of web agents, 2025. URL <https://arxiv.org/abs/2510.03204>.
- Kil, J., Song, C. H., Zheng, B., Deng, X., Su, Y., and Chao, W.-L. Dual-view visual contextualization for web navigation, 2024. URL <https://arxiv.org/abs/2402.04476>.
- Kim, G., Baldi, P., and McAleer, S. Language models can solve computer tasks, 2023. URL <https://arxiv.org/abs/2303.17491>.
- Koh, J. Y., Lo, R., Jang, L., Duvvur, V., Lim, M. C., Huang, P.-Y., Neubig, G., Zhou, S., Salakhutdinov, R., and Fried, D. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks, 2024. URL <https://arxiv.org/abs/2401.13649>.
- Koh, J. Y., McAleer, S., Fried, D., and Salakhutdinov, R. Tree search for language model agents, 2025. URL <https://arxiv.org/abs/2407.01476>.
- Kuntz, T., Duzan, A., Zhao, H., Croce, F., Kolter, Z., Flammarion, N., and Andriushchenko, M. Os-harm: A benchmark for measuring safety of computer use agents, 2025. URL <https://arxiv.org/abs/2506.14866>.

- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Lai, H., Liu, X., Iong, I. L., Yao, S., Chen, Y., Shen, P., Yu, H., Zhang, H., Zhang, X., Dong, Y., and Tang, J. Autowebglm: A large language model-based web navigating agent, 2024. URL <https://arxiv.org/abs/2404.03648>.
- Lee, D., Lee, J., Kim, K., Tack, J., Shin, J., Teh, Y. W., and Lee, K. Learning to contextualize web pages for enhanced decision making by llm agents, 2025. URL <https://arxiv.org/abs/2503.10689>.
- Lee, K.-H., Chen, X., Furuta, H., Canny, J., and Fischer, I. A human-inspired reading agent with gist memory of very long contexts, 2024. URL <https://arxiv.org/abs/2402.09727>.
- Li, T., Li, G., Deng, Z., Wang, B., and Li, Y. A zero-shot language agent for computer control with structured reflection, 2023. URL <https://arxiv.org/abs/2310.08740>.
- Liao, Z., Jones, J., Jiang, L., Ning, Y., Fosler-Lussier, E., Su, Y., Lin, Z., and Sun, H. Redteamcua: Realistic adversarial testing of computer-use agents in hybrid web-os environments, 2026. URL <https://arxiv.org/abs/2505.21936>.
- Liu, G., Geng, S., Li, S., Cui, H., Zhang, S., Liu, X., and Liu, T. Webcoach: Self-evolving web agents with cross-session memory guidance, 2025. URL <https://arxiv.org/abs/2511.12997>.
- Liu, X., Zhang, T., Gu, Y., Iong, I. L., Xu, Y., Song, X., Zhang, S., Lai, H., Liu, X., Zhao, H., Sun, J., Yang, X., Yang, Y., Qi, Z., Yao, S., Sun, X., Cheng, S., Zheng, Q., Yu, H., Zhang, H., Hong, W., Ding, M., Pan, L., Gu, X., Zeng, A., Du, Z., Song, C. H., Su, Y., Dong, Y., and Tang, J. Visualagentbench: Towards large multimodal models as visual foundation agents, 2024. URL <https://arxiv.org/abs/2408.06327>.
- Lù, X. H. and Reddy, S. Structured distillation of web agent capabilities enables generalization, 2026. URL <https://arxiv.org/abs/2604.07776>.
- Luo, T., Logeswaran, L., Johnson, J., and Lee, H. Visual test-time scaling for gui agent grounding, 2025. URL <https://arxiv.org/abs/2505.00684>.
- Lù, X. H., Kasner, Z., and Reddy, S. Weblinx: Real-world website navigation with multi-turn dialogue, 2024. URL <https://arxiv.org/abs/2402.05930>.
- Marino, K. and Marasović, A. Computer use survey: A visual survey of computer use agents, 2025. URL <https://kennethmarino.com/computeruse/computeruse.html>.
- Marreed, S., Oved, A., Yaeli, A., Shlomov, S., Levy, I., Akrabi, O., Sela, A., Adi, A., and Mashkif, N. Towards enterprise-ready computer using generalist agent, 2025. URL <https://arxiv.org/abs/2503.01861>.
- Miyai, A., Zhao, Z., Egashira, K., Sato, A., Sunada, T., Onohara, S., Yamanishi, H., Toyooka, M., Nishina, K., Maeda, R., Aizawa, K., and Yamasaki, T. Webchorearena: Evaluating web browsing agents on realistic tedious web tasks, 2025. URL <https://arxiv.org/abs/2506.01952>.
- Moravec, H. *Mind Children: The Future of Robot and Human Intelligence*. Harvard University Press, Cambridge, MA, 1988.
- Müller, M. and Žunic., G. Browser use: Enable ai to control your browser, 2024. URL <https://github.com/browser-use/browser-use>.
- Nekoei, H., Jaiswal, A., Bechard, P., Shliazhko, O., Ayala, O. M., Reymond, M., Caccia, M., Drouin, A., Chandar, S., and Lacoste, A. Just-in-time episodic feedback hinter: Leveraging offline knowledge to improve llm agents adaptation, 2025. URL <https://arxiv.org/abs/2510.04373>.
- OpenAI. Introducing operator, 2025. URL <https://openai.com/index/introducing-operator/>.

- Ouyang, S., Yan, J., Hsu, I.-H., Chen, Y., Jiang, K., Wang, Z., Han, R., Le, L. T., Daruki, S., Tang, X., Tirumalashetty, V., Lee, G., Rofouei, M., Lin, H., Han, J., Lee, C.-Y., and Pfister, T. Reasoningbank: Scaling agent self-evolving with reasoning memory, 2025. URL <https://arxiv.org/abs/2509.25140>.
- Pahuja, V., Lu, Y., Rosset, C., Gou, B., Mitra, A., Whitehead, S., Su, Y., and Awadallah, A. Explorer: Scaling exploration-driven web trajectory synthesis for multimodal web agents, 2025. URL <https://arxiv.org/abs/2502.11357>.
- Pang, X., Hong, R., Zhang, H., and Zhang, C. Assimilation and accommodation: Task-adaptive hierarchical abstraction for solving web tasks. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 14000–14014, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.720. URL <https://aclanthology.org/2025.findings-acl.720/>.
- Park, J., Tang, P., Das, S., Appalaraju, S., Singh, K. Y., Manmatha, R., and Ghadar, S. R-vm: Region-aware vision language model for precise gui grounding, 2025. URL <https://arxiv.org/abs/2507.05673>.
- Prabhu, V., Dai, Y., Fernandez, M., Gu, J., Ramakrishnan, K., Luo, Y., Savarese, S., Xiong, C., Li, J., Chen, Z., and Xu, R. Walt: Web agents that learn tools, 2025. URL <https://arxiv.org/abs/2510.01524>.
- Putkonen, A., Nioche, A., Laine, M., Kuuramo, C., and Oulasvirta, A. Fragmented visual attention in web browsing: Weibull analysis of item visit times. In Kamps, J., Goeuriot, L., Crestani, F., Maistro, M., Joho, H., Davis, B., Gurrin, C., Kruschwitz, U., and Caputo, A. (eds.), *Advances in Information Retrieval*, pp. 62–78, Cham, 2023. Springer Nature Switzerland. ISBN 978-3-031-28238-6.
- Sarch, G., Jang, L., Tarr, M. J., Cohen, W. W., Marino, K., and Fragkiadaki, K. Vlm agents generate their own memories: Distilling experience into embodied programs of thought, 2025a. URL <https://arxiv.org/abs/2406.14596>.
- Sarch, G., Saha, S., Khandelwal, N., Jain, A., Tarr, M. J., Kumar, A., and Fragkiadaki, K. Grounded reinforcement learning for visual reasoning, 2025b. URL <https://arxiv.org/abs/2505.23678>.
- ServiceNow. Browsergym leaderboard, 2025. URL <https://huggingface.co/spaces/ServiceNow/browsergym-leaderboard>.
- Shaw, P., Joshi, M., Cohan, J., Berant, J., Pasupat, P., Hu, H., Khandelwal, U., Lee, K., and Toutanova, K. From pixels to ui actions: Learning to follow instructions via graphical user interfaces, 2023. URL <https://arxiv.org/abs/2306.00245>.
- Shen, J., Jain, A., Xiao, Z., Amlekar, I., Hadji, M., Podolny, A., and Talwalkar, A. Scribeagent: Towards specialized web agents using production-scale workflow data, 2024. URL <https://arxiv.org/abs/2411.15004>.
- Shen, J., Bai, H., Zhang, L., Zhou, Y., Setlur, A., Tong, S., Caples, D., Jiang, N., Zhang, T., Talwalkar, A., and Kumar, A. Thinking vs. doing: Agents that reason by scaling test-time interaction, 2025. URL <https://arxiv.org/abs/2506.07976>.
- Singh, K., Singh, S., and Khanna, M. Trishul: Towards region identification and screen hierarchy understanding for large vlm based gui agents, 2025. URL <https://arxiv.org/abs/2502.08226>.
- Song, Y., Xu, F., Zhou, S., and Neubig, G. Beyond browsing: Api-based web agents, 2025. URL <https://arxiv.org/abs/2410.16464>.
- Su, H., Sun, R., Yoon, J., Yin, P., Yu, T., and Arik, S. O. Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments, 2025. URL <https://arxiv.org/abs/2501.10893>.

- Su, Y. Computer use: Modern moravec’s paradox. Yu’s Substack, September 2025. URL <https://yusu.substack.com/p/computer-use-modern-moravecs-paradox>. Blog post, accessed May 7, 2026.
- Tur, A. D., Meade, N., Lù, X. H., Zambrano, A., Patel, A., Durmus, E., Gella, S., Stańczak, K., and Reddy, S. Safearena: Evaluating the safety of autonomous web agents, 2025. URL <https://arxiv.org/abs/2503.04957>.
- Verma, G., Kaur, R., Srishankar, N., Zeng, Z., Balch, T., and Veloso, M. Adaptagent: Adapting multimodal web agents with few-shot learning from human demonstrations, 2024. URL <https://arxiv.org/abs/2411.13451>.
- Wang, Z., Wu, Q., Zhang, X., Zhang, C., Yao, W., Faisal, F. E., Peng, B., Qin, S., Nath, S., Lin, Q., Bansal, C., Zhang, D., Rajmohan, S., Gao, J., and Yao, H. Webxskill: Skill learning for autonomous web agents, 2026. URL <https://arxiv.org/abs/2604.13318>.
- Wang, Z. Z., Mao, J., Fried, D., and Neubig, G. Agent workflow memory, 2024. URL <https://arxiv.org/abs/2409.07429>.
- Wang, Z. Z., Gandhi, A., Neubig, G., and Fried, D. Inducing programmatic skills for agentic tasks, 2025. URL <https://arxiv.org/abs/2504.06821>.
- Wu, C. H., Shah, R., Koh, J. Y., Salakhutdinov, R., Fried, D., and Raghunathan, A. Dissecting adversarial robustness of multimodal llm agents, 2025. URL <https://arxiv.org/abs/2406.12814>.
- Wu, X., Hong, G., Chen, Y., Liu, M., Jin, F., Pan, X., Dai, J., and Liu, B. When bots take the bait: Exposing and mitigating the emerging social engineering attack in web automation agent, 2026. URL <https://arxiv.org/abs/2601.07263>.
- Xiang, Z., Zheng, L., Li, Y., Hong, J., Li, Q., Xie, H., Zhang, J., Xiong, Z., Xie, C., Yang, C., Song, D., and Li, B. Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning, 2025. URL <https://arxiv.org/abs/2406.09187>.
- Xu, H., Zhang, X., Liu, H., Wang, J., Zhu, Z., Zhou, S., Hu, X., Gao, F., Cao, J., Wang, Z., Chen, Z., Liao, J., Zheng, Q., Zeng, J., Xu, Z., Bai, S., Lin, J., Zhou, J., and Yan, M. Mobile-agent-v3.5: Multi-platform fundamental gui agents, 2026. URL <https://arxiv.org/abs/2602.16855>.
- Xu, Y., Lu, D., Shen, Z., Wang, J., Wang, Z., Mao, Y., Xiong, C., and Yu, T. Agenttrek: Agent trajectory synthesis via guiding replay with web tutorials, 2025. URL <https://arxiv.org/abs/2412.09605>.
- Xue, T., Qi, W., Shi, T., Song, C. H., Gou, B., Song, D., Sun, H., and Su, Y. An illusion of progress? assessing the current state of web agents, 2025. URL <https://arxiv.org/abs/2504.01382>.
- Yang, K., Liu, Y., Chaudhary, S., Fakoor, R., Chaudhari, P., Karypis, G., and Rangwala, H. Agentoccam: A simple yet strong baseline for llm-based web agents, 2025. URL <https://arxiv.org/abs/2410.13825>.
- Ying, Z., Shao, Y., Gan, J., Xu, G., Zhang, W., Zou, Q., Shi, J., Yin, Z., Zhang, M., Liu, A., and Liu, X. Securewebarena: A holistic security evaluation benchmark for lvlm-based web agents, 2026. URL <https://arxiv.org/abs/2510.10073>.
- Yu, S., Li, G., Shi, W., and Qi, P. Polyskill: Learning generalizable skills through polymorphic abstraction, 2025. URL <https://arxiv.org/abs/2510.15863>.
- Z.ai. Glm-4-32b-0414, 2025. URL <https://huggingface.co/zai-org/GLM-4-32B-0414>.
- Zhang, A. L., Kraska, T., and Khattab, O. Recursive language models, 2026. URL <https://arxiv.org/abs/2512.24601>.
- Zhang, K., Tenenholz, M., Polley, K., Ma, J., Yarats, D., and Li, N. Browsesafe: Understanding and preventing prompt injection within ai browser agents, 2025a. URL <https://arxiv.org/abs/2511.20597>.

- Zhang, R., Qiu, M., Tan, Z., Zhang, M., Lu, V., Peng, J., Xu, K., Agudelo, L. Z., Qian, P., and Chen, T. Symbiotic cooperation for web agents: Harnessing complementary strengths of large and small llms, 2025b. URL <https://arxiv.org/abs/2502.07942>.
- Zhang, Y., Ma, Z., Ma, Y., Han, Z., Wu, Y., and Tresp, V. Webpilot: A versatile and autonomous multi-agent system for web task execution with strategic exploration, 2024. URL <https://arxiv.org/abs/2408.15978>.
- Zhang, Y., Yu, T., and Yang, D. Attacking vision-language computer agents via pop-ups, 2025c. URL <https://arxiv.org/abs/2411.02391>.
- Zheng, B., Gou, B., Kil, J., Sun, H., and Su, Y. Gpt-4v(ision) is a generalist web agent, if grounded, 2024. URL <https://arxiv.org/abs/2401.01614>.
- Zheng, B., Fatemi, M. Y., Jin, X., Wang, Z. Z., Gandhi, A., Song, Y., Gu, Y., Srinivasa, J., Liu, G., Neubig, G., and Su, Y. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025a. URL <https://arxiv.org/abs/2504.07079>.
- Zheng, B., Liao, Z., Salisbury, S., Liu, Z., Lin, M., Zheng, Q., Wang, Z., Deng, X., Song, D., Sun, H., and Su, Y. Webguard: Building a generalizable guardrail for web agents, 2025b. URL <https://arxiv.org/abs/2507.14293>.
- Zhong, H., Faisal, F., França, L., Leesatapornwongsa, T., Szekeres, A., Rong, K., and Nath, S. Actionengine: From reactive to programmatic gui agents via state machine memory, 2026. URL <https://arxiv.org/abs/2602.20502>.
- Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. Webarena: A realistic web environment for building autonomous agents, 2024. URL <https://arxiv.org/abs/2307.13854>.

A Implementation Details

A.1 PageMem

We provide the details of our memory structure and describe the PageMem construction process. Each PageMem is built in two stages: DIVIDEPAGE (Algorithm 1) recursively partitions the live DOM tree into an ordered list of empty PageSections forming the structural skeleton of the page, and UPDATEPAGEMEM (Algorithm 2) then populates each section with its interactable elements and an LLM-generated summary, and finally generates the page-level summary.

A.1.1 Memory Structure

We provide a detailed formalization of the memory hierarchy from §2.2. Throughout, we use σ for model-generated summaries, α for immutable DOM-derived attributes, and μ for mutable agent-side state accumulated during exploration and task execution. Descriptions of α and μ below give representative fields rather than exhaustive listings; full field schemas are documented in our released code.

WebsiteMem. A WebsiteMem for website w is a tuple

$$\mathcal{M}_w = (P_w, T_w, E_w),$$

where P_w is a mapping from URL to PageMem, collecting all concrete pages encountered on w ; T_w is a list of list-page templates, each itself a PageMem, against which newly visited pages are matched by structural comparison (see A.2); and E_w is the set of all elements encountered on w , used to deduplicate elements during exploration.

PageMem. A PageMem is a tuple

$$p = (u_p, n_p, \sigma_p, S_p, \mu_p),$$

where u_p is the page URL; n_p is the page title; σ_p is a VLM-generated page-level summary; $S_p = (s_1, \dots, s_{|S_p|})$ is an ordered list of PageSections; and μ_p holds page-level agent state (e.g., information extracted by agent, the agent’s past interaction history on the page).

PageSection. A PageSection is a tuple

$$s = (\sigma_s, E_s, S'_s, \alpha_s, \mu_s),$$

where σ_s is a VLM-generated section summary; $E_s = (e_1, \dots, e_{k_s})$ is an ordered list of Elements contained in the section; $S'_s = (s'_1, \dots, s'_{m_s})$ is an ordered list of sub-sections, empty for normal sections and containing one sub-section per item for list sections; α_s holds DOM-derived attributes used for creating selectors (e.g., id, tag, class, DOM-subtree handle, bounding box) and μ_s holds mutable agent state (e.g., task-relevant extractions, VLM-generated image descriptions, a staleness flag indicating whether the DOM subtree has changed since σ_s was last computed).

Element. An Element is a tuple

$$e = (\alpha_e, E'_e, \mu_e),$$

where α_e holds DOM-derived attributes (e.g., id, tag, class, role, label, type); $E'_e = (e'_1, \dots, e'_{l_e})$ is an ordered list of dropdown items (themselves Elements), which is empty for non-dropdown elements; and μ_e holds mutable agent state (e.g., the element's current input value, a flag for whether the agent has clicked the element during the current task).

A.1.2 Page division.

We provide the pseudocode for page splitting in Algorithm 1. DIVIDEPAGE takes the root of the DOM tree and returns a PageMem whose ordered section list forms the structural skeleton of the page. The procedure recursively descends the DOM, terminating at nodes that form a meaningful grouping, either semantically (by tag), visually (by size), or structurally (by repetition of siblings). Groups of ≥ 4 consecutive siblings sharing tag and class are merged into a single list section node before recursion; list sections are always terminal and are never further subdivided.

Algorithm 1 DIVIDEPAGE

Input: DOM root node r

Output: PageMem p

```

1:  $L \leftarrow []$ 
2: SPLIT( $r, L$ )
3:  $p \leftarrow \text{NEWPAGEMEM}()$ 
4:  $p.u \leftarrow \text{CURRENTURL}()$ 
5:  $p.n \leftarrow \text{EXTRACTTITLE}()$ 
6:  $p.S \leftarrow L$ 
7: return  $p$ 

8: procedure SPLIT(node  $v$ , list  $L$ )
9:   if ISTERMINAL( $v$ ) then
10:     append MAKESECTION( $v$ ) to  $L$ 
11:   else
12:      $C \leftarrow \text{GROUPSIBLINGS}(v.\text{children})$ 
13:     for  $c \in C$  do
14:       SPLIT( $c, L$ )

15: function ISTERMINAL(node  $v$ )
16:   return  $v.\text{isListSection} \vee v.\text{tag} \in \mathcal{T}_{\text{group}} \vee \neg \text{OVERSIZED}(v)$ 

17: function OVERSIZED(node  $v$ )
18:   return  $(v.h > 900 \wedge v.w > 320) \vee (v.h > 500 \wedge v.w > 800)$ 

19: function GROUPSIBLINGS(children  $(c_1, \dots, c_k)$ )
20:   scan  $(c_1, \dots, c_k)$  for groups of consecutive siblings that share tag and class
21:   replace each group of length  $\geq 4$  with a single list-section node containing the group
22:   return the resulting (shortened) sequence

```

Parameters. The grouping tag set is

$$\mathcal{T}_{\text{group}} = \{\text{ol}, \text{ul}, \text{table}, \text{form}, \text{fieldset}, \text{aside}, \text{article}, \text{details}, \text{p}, \text{img}, \text{embed}, \text{code}, \text{group}, \text{nav}, \text{header}, \text{footer}\}.$$

Dimensions $v.h$ and $v.w$ in OVERSIZED are the node's rendered bounding-box height and width in CSS pixels, obtained from the browser's layout engine.

A.1.3 PageMem update.

UPDATEPAGEMEM refreshes a PageMem to reflect the current live page state and is invoked at the start of every observation step. It is also the routine that populates a freshly divided PageMem with its initial elements and summaries.

UPDATESECTION (i) queries the browser for the section's current set of interactable elements, (ii) computes the added / removed / modified diff $\Delta = (\Delta^+, \Delta^-, \Delta^\sim)$ against the section's previous element list, (iii) re-summarizes if the section has no summary yet or if the structural change is large enough, and (iv) returns Δ so that the caller (an observation step or a workflow) can respond to partial state changes. This is the machinery behind, e.g., the dropdown workflow of §2.5, which reads the revealed options directly from Δ^+ . List sections are handled specially: their content is unbounded and repetitive, so instead of enumerating elements or tracking a diff at the list level, we always re-summarize from a screenshot and return an empty diff. Element-level tracking happens only on the per-item sub-sections, and only after list-item selection in the observation pipeline (§2.4).

Algorithm 2 UPDATEPAGEMEM and UPDATESECTION

```

1: procedure UPDATEPAGEMEM(PageMem  $p$ )
2:   for  $s \in p.S$  do
3:     UPDATESECTION( $s$ )
4:   if  $p.\sigma_p$  is undefined then
5:      $p.\sigma_p \leftarrow \text{VLMSUMMARIZEPAGE}(p)$ 

6: procedure UPDATESECTION(PageSection  $s$ )
7:   if  $s$  is a list section then                                ▷ list sections only get summarized
8:      $s.\sigma_s \leftarrow \text{VLMSUMMARIZESECTION}(s)$ 
9:     return  $(\emptyset, \emptyset, \emptyset)$ 
10:   $E_{\text{new}} \leftarrow \text{GETELEMENTS}(s)$ 
11:   $\Delta \leftarrow \text{DIFF}(s.E_s, E_{\text{new}})$                                 ▷  $\Delta^+$ : added,  $\Delta^-$ : removed,  $\Delta^\sim$ : input-value changes
12:   $s.E_s \leftarrow E_{\text{new}}$ 
13:  if  $s.\sigma_s$  is undefined then
14:     $s.\sigma_s \leftarrow \text{VLMSUMMARIZESECTION}(s)$ 
15:  else if  $|\Delta^+| + |\Delta^-| \geq 3$  then                                ▷ re-summarize if  $\geq 3$  elements added/removed
16:     $s.\sigma_s \leftarrow \text{VLMSUMMARIZESECTION}(s)$ 
17:  return  $\Delta$ 

```

Element population. GETELEMENTS produces the element list for a section via a three-step pipeline. A helper first resolves the section to a Playwright locator. The locator's descendants are then filtered by the clickable predicate ISCLICKABLE defined below. Finally, each surviving DOM node is passed to an Element constructor that reads its DOM attributes into α_e .

Clickable predicate. A DOM node v is considered interactable iff it passes a visibility-and-accessibility gate *and* satisfies at least one positive signal. The gate excludes nodes that are not rendered, carry the disabled attribute, or have `aria-hidden="true"`. The positive signals are any of: a tag in an interactable tag set, a DOM event-listener attribute in a listener set, an ARIA role in an interactable role set, or a computed cursor style of pointer. Formally,

$$\text{ISCLICKABLE}(v) \equiv (v.\text{tag} \in \mathcal{T}_{\text{clk}} \vee v.\text{attrs} \cap \mathcal{L}_{\text{clk}} \neq \emptyset \vee v.\text{role} \in \mathcal{R}_{\text{clk}} \vee v.\text{cursor} = \text{pointer}) \wedge \text{ACCESSIBLE}(v),$$

with the sets

$$\begin{aligned}\mathcal{T}_{\text{clk}} &= \{\text{button, a, input, select, textarea, details, summary, option}\}, \\ \mathcal{L}_{\text{clk}} &= \{\text{onclick, onmousedown, onmouseup, onkeydown, onkeyup}\}, \\ \mathcal{R}_{\text{clk}} &= \{\text{button, link, menuitem, option, radio, checkbox, tab,} \\ &\quad \text{textbox, combobox, slider, spinbutton, search, searchbox}\}.\end{aligned}$$

These heuristics are adapted from BrowserUse (Müller & Žunic., 2024).

A.2 Exploration

We provide the details of the offline exploration procedure that builds the WebsiteMem $\mathcal{M}_w$ used at inference. Exploration is a deterministic depth-first traversal of a website’s pages and clickable elements, deduplicated against the running set E_w of all elements seen on the site, with state restored between element clicks by reloading the pre-click URL. EXPLOREPAGE (Algorithm 3) is the recursive driver that visits one page at a time; it delegates to ITERATEPAGE and EXPLOREELEMENT (Algorithm 4) for the element-level work. Exploration is launched per website by initializing an empty $\mathcal{M}_w$ and a URL-only PageMem stub at a chosen starting URL — the homepage in our experiments — and invoking EXPLOREPAGE at the configured maximum depth. We abstract over the per-page element budget, total page budget, and per-website timeout in the pseudocode for clarity; these limits act as additional early-return checks throughout, and their values are reported per benchmark in Appendix B.

Algorithm 3 EXPLOREPAGE

```

1: procedure EXPLOREPAGE(PageMem stub  $p$ , depth  $d$ )
2:   if  $p.u \in \mathcal{M}_w.P_w$  then                                ▷ URL already explored on this site
3:     return
4:   NAVIGATE( $p.u$ )
5:   Populate  $p$  with sections, elements and summaries
6:   add  $p$  to  $\mathcal{M}_w.P_w$  keyed by  $p.u$ 
7:   if MATCHESTEMPLATE( $p$ ,  $\mathcal{M}_w.T_w$ ) then                    ▷ already explored page with same structure
8:     return
9:   else if HASLISTSECTION( $p$ )  $\vee$   $p.is\_list\_item$  then
10:    add  $p$  to  $\mathcal{M}_w.T_w$ 
11:     $N \leftarrow$  ITERATEPAGE( $p$ )                                ▷ explore elements on page
12:    if  $d = 0$  or budget exhausted then return
13:    for  $p' \in N$  do
14:      EXPLOREPAGE( $p'$ ,  $d - 1$ )

```

Page-level traversal. EXPLOREPAGE takes a stub PageMem carrying a target URL and the remaining recursion depth. It deduplicates against URLs already in $\mathcal{M}_w$, navigates the browser to the page, runs DIVIDEPAGE on the freshly-loaded DOM to construct the full PageMem, and UPDATEPAGE-MEM to populate elements and summaries. The newly-built PageMem is then registered in $\mathcal{M}_w$. If its section structure matches an existing template in T_w , the page is treated as a known-shape duplicate and not iterated, since further iteration would re-cover element behaviors already learned from the matching template; otherwise, if the page contains a list section or its `is_list_item` flag is set, the PageMem is added to T_w as a new template. ITERATEPAGE is then called on the page, returning a list of stub PageMems for newly-discovered URLs, which the procedure recursively explores at depth $d - 1$.

Template matching. MATCHESTEMPLATE compares the candidate PageMem against each template in T_w . Two PageMems match when they have the same number of sections and each pair of corresponding sections is structurally equivalent under the DOM-derived attributes in α_s (tag, class, and other selector-defining attributes). Because DIVIDEPAGE is deterministic on the DOM, structurally equivalent pages reliably yield identical section sequences in practice, so exact structural equality is sufficient as a match criterion without needing a similarity threshold. Matching is checked only against T_w rather than all of P_w , both for efficiency and because non-template pages are by definition idiosyncratic and not expected to recur.

Element-level traversal. ITERATEPAGE walks the page’s elements in document order. For elements not contained in any list section, it skips those already in the global element set E_w and registers each new element in E_w . List sections are handled separately: rather than iterating every list item (which would redundantly re-cover elements with structurally identical neighbors), the procedure invokes ITERATELISTITEM, which iterates the elements contained in a single list-item container using the same per-element logic. Stubs returned from list-item exploration are tagged with `is_list_item` so that the recursive EXPLOREPAGE call can promote the resulting pages to templates.

EXPLOREELEMENT returns the newly-discovered URL stub(s) reached by clicking the element. It first applies a static skip filter (described below) that rules out elements unsafe or unhelpful to click. It then records the pre-click URL, clicks the element, and inspects the result. If the URL has not changed, the post-click diff in page state is computed. Any newly-revealed elements (Δ^+) are recorded as the clicked element’s `dropdown_elements` and recursively explored using the same EXPLOREELEMENT routine. If the URL has changed and points to a same-site page not yet in $\mathcal{M}_w$, a URL-only stub is created and returned. Finally, the browser is reloaded to the pre-click URL to restore page state for the next iteration.

Algorithm 4 ITERATEPAGE and EXPLOREELEMENT

```

1: procedure ITERATEPAGE(PageMem  $p$ )
2:    $N \leftarrow []$ 
3:   for element  $e$  in  $p$  not contained in a list section do
4:     if per-page element budget exhausted then break
5:     if  $e \in \mathcal{M}_w.E_w$  then continue
6:     add  $e$  to  $\mathcal{M}_w.E_w$ 
7:      $N \leftarrow N + \text{EXPLOREELEMENT}(e)$ 
8:   for list section  $s \in p.S$  do
9:      $N_\ell \leftarrow \text{ITERATELISTITEM}(s)$  ▷ explore elements in one list-item container
10:    for  $p' \in N_\ell$  do  $p'.\text{is\_list\_item} \leftarrow \text{true}$ 
11:     $N \leftarrow N + N_\ell$ 
12:   return  $N$ 

13: procedure EXPLOREELEMENT(Element  $e$ )
14:    $N \leftarrow []$ 
15:   if SHOULDSKIP( $e$ ) then return  $N$ 
16:    $u_{\text{pre}} \leftarrow \text{CURRENTURL}()$ 
17:   CLICK( $e$ )
18:    $u_{\text{post}} \leftarrow \text{CURRENTURL}()$ 
19:   if  $u_{\text{post}} = u_{\text{pre}}$  then
20:     Identify newly revealed elements  $\Delta^+$ 
21:     if  $\Delta^+ \neq \emptyset$  then ▷ click revealed new elements (dropdown opened)
22:        $e.\text{dropdown\_elements} \leftarrow \Delta^+$ 
23:       for  $e' \in \Delta^+$  do
24:          $N \leftarrow N + \text{EXPLOREELEMENT}(e')$  ▷ explore dropdown elements
25:     else if  $u_{\text{post}}$  is on the same site  $w$  and  $u_{\text{post}} \notin \mathcal{M}_w.P_w$  then
26:       create stub  $p_{\text{new}}$  with  $p_{\text{new}}.u \leftarrow u_{\text{post}}$ 
27:        $N \leftarrow N + [p_{\text{new}}]$ 
28:   NAVIGATE( $u_{\text{pre}}$ ) ▷ restore pre-click state for the next iteration
29:   return  $N$ 

```

Skip filter. SHOULDSKIP excludes four categories of elements before any click is issued: (i) off-site links, identified by an href pointing to a domain outside w ; (ii) authentication links such as login and sign-up, identified by keyword matching against the link text and URL path; (iii) `tel:`, `mailto:`, and `javascript:print(. . .)` links, identified by the href scheme; and (iv) *modifier* buttons that could mutate persistent site state, identified either by the form attribute `type="submit"` or by keyword matching of the element’s accessible text against destructive terms (delete, remove, submit, save, etc.).

A.3 Observation Pipeline

We provide the details of the detail-extraction and summarization stages of the observation pipeline (§2.4). ANALYZEPAGE (Algorithm 5) acts as the main driver: given the set of sections S_t selected as relevant, it extracts task-relevant information from each and synthesizes a page summary. List sections are routed through SELECTLISTITEMS (Algorithm 6), which uses chunked LLM selection with explicit early termination to keep arbitrarily long lists within context.

Per-section detail extraction. For each selected section, a helper FORMAT produces the *details string* consumed by the extraction LLM. For a normal section, the details string contains the section’s accessibility subtree together with the URLs and VLM-generated descriptions of any images in the section above the minimum size threshold (50 x 50 pixels). For a list section, SELECTLISTITEMS is invoked first to choose a subset of items, and the details string is the per-item content formatted as a numbered list, with each entry containing the same accessibility-subtree-plus-image content as a normal section. The extraction call LLMEXTRACTDETAILS (Prompt E.1) caches its output on the PageSection together with the details string used to produce it; on a subsequent call with an identical details string, the cached extraction is returned without an LLM call.

Algorithm 5 ANALYZEPAGE

```

1: procedure ANALYZEPAGE(PageMem  $p$ , selected sections  $S_t$ )
2:    $X \leftarrow []$  ▷ per-section extraction strings, in selection order
3:   for  $s \in S_t$  do
4:     if  $s$  is a list section then
5:       SELECTLISTITEMS( $s$ ) ▷ populates  $s$ ’s sub-sections and  $s.E_s$ 
6:        $D \leftarrow \text{FORMAT}(s)$ 
7:        $x \leftarrow \text{LLMEXTRACTDETAILS}(D)$  ▷ returns cached value if  $D$  unchanged
8:       append " $\langle \text{id} \rangle \langle \text{tag} \rangle \langle \text{class} \rangle$ :" +  $x$  to  $X$ 
9:    $p.\text{task\_summary} \leftarrow \text{LLMSUMMARIZEPAGE}(X)$  ▷ regenerated every call
10:  return  $p.\text{task\_summary}$ 

```

List item selection. A list section can contain hundreds or thousands of items, far exceeding what fits in a single LLM context. SELECTLISTITEMS addresses this by chunking the items sequentially into fixed-size groups and prompting the LLM to select relevant items chunk by chunk (Prompt E.1.1). After each chunk, a separate LLM call is issued (Prompt E.1.1) that sees the indices already searched, the items already selected, and the remaining entries, and decides whether to terminate early — this avoids paying the cost of scanning the full list when the relevant items have already been found (e.g., the top few results of a sorted list). After selection, the procedure rebuilds the list section’s sub-sections from the selected items, populating each via UPDATESECTION, and overwrites the list section’s element list E_s with only the elements from selected items. The original full element set is not retained: re-selection on a later observation step rebuilds the sub-sections from scratch from the live page state.

Algorithm 6 SELECTLISTITEMS

```

1: procedure SELECTLISTITEMS(list section  $s$ )
2:    $I^* \leftarrow []$  ▷ indices of items selected so far
3:   partition the items of  $s$  into sequential chunks  $(C_1, C_2, \dots)$  of fixed size  $c$ 
4:   for  $k = 1, 2, \dots$  do
5:      $I^* \leftarrow I^* + \text{LLMSELECTITEMS}(C_k, I^*)$ 
6:     if LLMCHECKDONE( $I^*$ , indices searched so far, remaining items) then
7:       break
8:   rebuild  $s.S'_s$  as one PageSection per item index in  $I^*$ 
9:   for  $s' \in s.S'_s$  do UPDATESECTION( $s'$ )
10:   $s.E_s \leftarrow$  concatenation of  $s'.E_{s'}$  over  $s' \in s.S'_s$  ▷ only selected items contribute actions

```

Summary caching. Two caches operate at different lifetimes within the observation pipeline. Section summaries σ_s are populated by UPDATESECTION (Algorithm 2) and persist across tasks

within a WebsiteMem. Per-section task extractions x produced by LLMEXTRACTDETAILS are cached on the PageSection alongside the details string D that produced them, and are reused for the lifetime of a task whenever the section’s content is unchanged. The page-level task summary $p.task_summary$ is always regenerated on each call to ANALYZEPAGE, since the relevant framing of a page can shift as the task progresses through its history h_t .

A.4 Agent Loop

We provide the details of the top-level inference loop that integrates the observation pipeline (§2.4, App. A.3) with the action system (§2.5). AGENTLOOP (Algorithm 7) executes one timestep at a time until either the task is verified complete or the step budget is exhausted. Each timestep produces *one* agent action, which may itself be a compound workflow that internally issues multiple LLM sub-calls and browser operations. After a non-navigating action, an intra-step continuation loop allows the agent to chain follow-up actions on the same page without re-running the full observation pipeline, up to a small budget. Bookmark and (where applicable) website pre-selection happen once at task start (Prompt E.2.1).

Algorithm 7 AGENTLOOP

```

1: procedure AGENTLOOP(task  $\tau = (I, u_0)$ , WebsiteMem  $\mathcal{M}_w$ )
2:    $B_\tau \leftarrow \text{LLMSELECTBOOKMARKS}(\mathcal{M}_w, I)$  ▷ optional, at task start only
3:    $h \leftarrow []$ 
4:   for  $t = 1, \dots, T_{\max}$  do
5:      $p \leftarrow \text{GETPAGEMEM}(\text{current URL}, \mathcal{M}_w)$ 
6:     if  $m \leftarrow \text{CHECKMODAL}(p)$  then
7:        $S \leftarrow [m]$  ▷ modal: focus on dialog, skip section selection
8:     else
9:        $S \leftarrow \text{LLMSELECTSECTIONS}(p)$ 
10:     $\hat{o} \leftarrow \text{ANALYZEPAGE}(p, S)$ 
11:     $u_{\text{pre}} \leftarrow \text{current URL}$ 
12:    for  $j = 1, \dots, J_{\max}$  do ▷ intra-step continuation,  $J_{\max} = 5$ 
13:       $\mathcal{A} \leftarrow \text{GATHERCANDIDATES}(p, S, p.S \setminus S, B_\tau)$ 
14:       $(a, r) \leftarrow \text{LLMSELECTACTION}(\mathcal{A})$  ▷ up to 3 retries on action error
15:      if  $a$  is end-task then
16:        if  $\text{LLMVERIFYENDTASK}(I, h)$  then ▷ 1 verification check per task
17:          return  $\text{LLMFINALANSWER}(I, h)$ 
18:        else
19:          remove end-task from  $\mathcal{A}$  and re-prompt for  $a$ 
20:         $\text{EXECUTEACTION}(a)$  ▷ single op or compound workflow
21:        if current URL  $\neq u_{\text{pre}}$  then break
22:        if  $\text{CHECKMODAL}(p) \neq \text{nil}$  then break
23:         $\text{UPDATEPAGEMEM}(p)$ 
24:         $\hat{o}' \leftarrow \hat{o} + \text{VLMSCREENDIFF}(p)$  ▷ cheap update for follow-up action
25:        append step observation, reason, and action to  $h$ 
26:    return  $\text{LLMFINALANSWER}(I, h)$  ▷ step budget exhausted

```

Observation phase. At the start of each timestep, the agent retrieves or constructs the PageMem p_t for the current page and refreshes it via UPDATEPAGEMEM. A modal-detection helper CHECKMODAL then tests for the presence of a modal dialog using DOM heuristics including `role="dialog"` and `aria-modal="true"`; when a modal is detected, section selection is bypassed entirely and the modal’s PageSection is used as the sole relevant section, focusing the agent’s attention on the dialog and preventing the surrounding (now-inert) page from polluting the candidate space. Otherwise, the LLM selects relevant sections S_t from the section summaries as in §2.4 (Prompt E.1). The remaining sections $S_t^c = p_t.S \setminus S_t$ are kept aside for use during candidate assembly. ANALYZEPAGE then produces the task summary $\hat{o}_t$.

Action phase. GATHERCANDIDATES assembles the candidate set $\mathcal{A}_t$ in two passes. First, for each $s \in S_t$, an LLM call selects elements from s likely to be useful for the task, conditioned on s 's extracted details (Prompt E.2.1). Second, a single LLM call covers the elements of the first five entries of S_t^C in document order — a heuristic that ensures upper-page UI (navigation bars, search boxes, primary buttons) remains reachable even when the LLM did not flag those sections as relevant during section selection. Navigation actions (visited URLs, bookmarks B_r , type-URL, switch-tab, switch-website) are filtered by an LLM pass against the task (Prompt E.2.1). The end-task action is always appended. A rule-based pre-filter removes irrelevant actions (e.g., switching to an already active tab, clicking an already-selected radio button, print, tel links, links leading outside allowed domains) before $\mathcal{A}_t$ is presented to the LLM.

The action-selection LLM call (Prompt E.2.1) returns a chosen action a together with a natural-language *reason* r explaining the choice. EXECUTEACTION dispatches to the workflow appropriate to the selected action's type. Navigation actions invoke a single NAVIGATE call; element actions inside a form section invoke SUBMITFORM, while other element actions invoke ELEMENTACTION, which routes to the appropriate workflow (App. A.5); the end-task action invokes LLMVERIFYENDTASK. If the chosen action raises a runtime error (an invalid URL, a stale selector, an interaction failure on a non-interactable element), the action is removed from $\mathcal{A}_t$ and the LLM is re-prompted; this retry budget resets each timestep and is bounded at three attempts.

End-task verification. When the LLM selects end-task, LLMVERIFYENDTASK issues an LLM call conditioned on the task instruction I and the interaction history h_t that judges whether the task has been completed (Prompt E.2.1). If completion is verified, a separate LLMFINALANSWER call (Prompt E.2.1) produces the answer string and the loop terminates. If not, the end-task action is removed from $\mathcal{A}_t$ for the current timestep only — it remains available on subsequent timesteps — and the LLM is re-prompted to choose a different action. We allow at most one verification check per task episode; subsequent end-task actions end the task immediately.

Intra-step continuation. Some actions (entering input into a field, copying to clipboard) leave the page on the same URL and complete a partial rather than full intent. To avoid the overhead of restarting the observation pipeline for follow-up actions on the same page, after such an action the loop enters a short continuation phase: UPDATEPAGEMEM refreshes the page state, a VLM is prompted to describe the visual difference and the description is concatenated with $\hat{o}_t$ to produce the updated observation $\hat{o}'_t$, and the LLM selects another action from a freshly-gathered candidate set. The continuation phase ends when either the page URL changes, a modal dialog appears (handled at the next timestep with focused attention), the agent selects end-task, or a budget of five within-step actions is reached.

A.5 Action Workflows

We provide the details of the workflows invoked by EXECUTEACTION when the agent selects an element action (App. A.4). Element actions are dispatched in two ways: if the selected element lies inside a form section, control passes to SUBMITFORM; otherwise it passes to ELEMENTACTION (Algorithm 8), which routes to the appropriate per-element-type workflow based on tag, role, and DOM attributes. Navigation actions and end-task are handled directly in the agent loop and are not covered here.

Element-type dispatch. ELEMENTACTION is structured as a flat decision tree over element properties: it checks first for behaviors known from exploration (recorded dropdown_elements), then for input-type-specific handlers (file upload, select/combobox, the various input/textarea subtypes), then for “probably opens something” signals (aria-haspopup, or any element not yet explored), and finally falls through to a plain click.

Form submission. SUBMITFORM (Algorithm 9) handles forms in three phases. An LLM call first selects which fields to fill (Prompt E.2.2), and ELEMENTACTION is invoked on each chosen field, dispatching to ENTERINPUT, SELECTOPTION, or UPLOADFILE as appropriate. A validation pass then re-fills any field that is empty-but-required or carries aria-invalid="true" after the initial entry — the LLM is re-prompted for new values for these fields. Finally, a review loop allows the LLM to inspect the populated form and either edit additional fields, submit, or exit

Algorithm 8 ELEMENTACTION

```
1: procedure ELEMENTACTION(Element  $e$ )
2:   if  $e.dropdown\_elements \neq \emptyset$  then ▷ recorded from exploration
3:     DROPDOWNACTION( $e$ )
4:   else if  $e.input\_type = file$  then
5:     UPLOADFILE( $e$ )
6:   else if  $e.tag = select \vee e.role = combobox$  then
7:     SELECTOPTION( $e$ )
8:   else if  $e.tag \in \{input, textarea\} \vee e.role = spinbutton$  then
9:     if  $e.input\_type \in \{submit, reset, button\}$  then
10:      CLICELEMENT( $e$ )
11:     else if  $e.input\_type = search$  then
12:      SEARCH( $e$ )
13:     else if  $e$  is a radio or checkbox then
14:      CLICELEMENT( $e$ )
15:     else
16:       ENTERINPUT( $e$ )
17:   else if  $e.aria-haspopup \neq false \vee \neg e.explored$  then ▷ probe for dropdown semantics
18:     DROPDOWNACTION( $e$ )
19:   else if  $e$  is a copy button then
20:     COPYTOCLIPBOARD( $e$ )
21:   else
22:     CLICELEMENT( $e$ )
```

(Prompt E.2.2); exiting leaves the form in its current state and returns control to the main agent loop without submitting. Within the review loop the candidate set is restricted to elements of the form section.

Algorithm 9 SUBMITFORM

```
1: procedure SUBMITFORM(form section  $s$ )
2:    $F \leftarrow \text{LLMSELECTFIELDS}(s)$ 
3:   for  $f \in F$  do
4:     ELEMENTACTION( $f$ )
5:     UPDATESECTION( $s$ )
6:   for  $f \in s.E_s$  where  $f$  is empty-and-required or  $f.aria-invalid = true$  do
7:     ELEMENTACTION( $f$ )
8:   UPDATESECTION( $s$ )
9:   for  $k = 1, \dots, K_{\max}$  do ▷ review phase,  $K_{\max} = 15$ 
10:     $a \leftarrow \text{LLMSELECTFORMACTION}(s)$  ▷ element in  $s$ , submit, or exit
11:    if  $a$  is exit or  $a$  is a submit button then
12:      if  $a$  is a submit button then CLICELEMENT( $a$ )
13:      return
14:    ELEMENTACTION( $a$ )
15:    UPDATESECTION( $s$ )
16:    if current URL has changed then return
```

Dropdown action. DROPDOWNACTION (Algorithm 10) clicks the dropdown trigger and consults the section diff returned by UPDATEPAGEMEM. Three outcomes are possible. If the URL changed or no new elements were revealed, the click was an ordinary navigation or null action and the workflow returns. If the revealed elements form a coherent form-like cluster (multiple inputs together with a submit-like button), control is routed to SUBMITFORM on the synthesized form section. Otherwise — the typical case of a menu, autocomplete list, or option dropdown — the LLM selects one of the revealed elements (Prompt E.2.1) and that element is clicked. The form-detection heuristic ISFORM returns true when Δ^+ contains at least two input-like elements and at least one element matching submit-button heuristics (a button with type submit, or accessible text matching submit-like keywords).

Algorithm 10 DROPDOWNACTION

```
1: procedure DROPDOWNACTION(element  $e$ )
2:    $u_{\text{pre}} \leftarrow$  current URL
3:   CLICKELEMENT( $e$ )
4:    $\Delta \leftarrow$  UPDATEPAGEMEM(current page)
5:   if current URL  $\neq u_{\text{pre}}$  or  $\Delta^+ = \emptyset$  then return
6:   if ISFORM( $\Delta^+$ ) then
7:     SUBMITFORM(synthesize form section from  $\Delta^+$ )
8:   else
9:      $e' \leftarrow$  LLMSELECTACTION( $\Delta^+$ )
10:    CLICKELEMENT( $e'$ )
```

Other workflows.

- **SEARCH**: invokes ENTERINPUT on the search field, then computes a section diff to detect whether suggestions have appeared. If they have, the LLM is offered the option to select a suggestion (which is then clicked) or to ignore them (Prompt E.2.2). The workflow concludes by pressing the Enter key to issue the search.
- **ENTERINPUT**: prompts the LLM for the value to enter (Prompt E.2.2) and fills it into the field.
- **UPLOADFILE**: presents the LLM with the choice of either an existing file in the agent’s local filesystem (input files staged for the task — e.g., the input images supplied with VisualWebArena tasks — and any text files created earlier in the same task or VLM-captioned images saved during the task) or a *create-new-file* option (Prompt E.2.2). In the latter case the agent is prompted for a filename and text content (Prompt E.2.2), the file is written to the local filesystem, and the new file is then uploaded.
- **SELETOPTION**: prompts the LLM to choose one of the available options (Prompt E.2.2) and sets the field’s value to that option via Playwright.
- **COPYTOCLIPBOARD**: reads the copied text from the clipboard and logs it in the action history.
- **CLICKELEMENT**: issues a Playwright click on the element.

A.5.1 Action Logging and History Format

Every successful basic action contributes a string to the agent’s interaction history. Failed actions (those that raise the runtime errors handled by the retry mechanism in App. A.4) are not logged — only the eventually-successful action appears. Action strings are constructed *after* execution, since several formats reference values that are known only post-execution (e.g., the actual text entered into a field, the option that was selected). Table 5 lists the format for each basic action.

Action	Log string format
CLICKELEMENT	Clicked {elem_name}
COPYTOCLIPBOARD	Copied text to clipboard: "{value}"
SELETOPTION	Selected value of "{option}" for field "{elem_name}"
ENTERINPUT	Entered value of "{input_value}" into {elem_name}
UPLOADFILE	Uploaded file: "{file}" into the "{elem_name}" field
SWITCHTAB	Switched from tab {start_i} ({start_url}) to tab {new_i} ({new_url})
SWITCHWEBSITE	Navigated to the "{website_name}" website ({website_url})
GO TOPAGE	Navigated to page "{page_name}" ({page_url})
GOBACK	Returned to the "{page_name}" page
ENTERURL	Navigated to URL: "{url}"

Table 5: Log string format for each basic action.

Compound actions and intra-step continuation chains produce multiple basic-action strings within a single timestep. These are emitted as a list under an **Actions:** heading in the history; a timestep

that produced exactly one basic action uses the singular Action: heading instead. Each timestep contributes a block of the form below to the history, with the page name and URL drawn from the PageMem, the task summary from ANALYZEPAGE, and the reason produced by the LLM in the same call as the action selection itself.

```
* Step 1:
  * Observation: {page_name} ({url})
    - Summary: {task_summary}
  * Reason for Action: {reason}
  * Action: {action_str}
* Step 2:
  * Observation: {page_name} ({url})
    - Summary: {task_summary}
  * Reason for Action: {reason}
  * Actions:
    - {action_str}
    - {action_str}
```

B Additional Experiment Details

Exploration parameters. Offline exploration is bounded by four limits per website: a maximum of 75 clickable elements explored per page, 500 pages per website, and a search depth of 2 (where the homepage is at depth 0, so a depth-2 traversal covers three layers). Each website is also subject to a 12-hour wall-clock timeout, after which exploration terminates and the partial WebsiteMem is used as-is. For Online-Mind2Web, which spans 136 distinct websites, depth is reduced to 1 and the per-website timeout to 1 hour. WorkArena uses the same parameters as WebArena and VisualWebArena.

URL replacement on WebArena and VisualWebArena. WebArena and VisualWebArena evaluate against locally-hosted simulated copies of real websites (Reddit, GitLab, OpenStreetMap, etc.), but task instructions refer to these sites by their real names. We observed that LLMs frequently misinterpret this as a directive to navigate to the real site — e.g., reading the localhost URL, concluding “I am not on Reddit,” and attempting to navigate to `https://www.reddit.com`, which breaks evaluation. We address this confusion with a bidirectional URL substitution applied at the prompt boundary: simulated-site URLs are rewritten to their real-site counterparts in every string passed to the LLM, and the inverse rewrite is applied to URLs in LLM outputs before they reach the browser. The LLM thus reasons consistently as if it were operating on the real site, while the browser remains pointed at the simulation. Table 6 lists the seven substitutions used.

Simulated host	Real-site URL
\$SHOPPING	<code>https://www.onestopmarket.com</code>
\$CLASSIFIEDS	<code>https://osclass-classifieds.com</code>
\$REDDIT	<code>https://www.reddit.com</code>
\$WIKIPEDIA	<code>https://en.wikipedia.org</code>
\$SHOPPING_ADMIN	<code>http://magento.site/</code>
\$GITLAB	<code>https://gitlab.com</code>
\$MAP	<code>http://openstreetmap.org</code>

Table 6: URL substitutions applied at the prompt boundary on WebArena and VisualWebArena. Environment variables hold the localhost URLs of the simulated sites. The substitution is applied bidirectionally: simulated $\rightarrow$ real on input to the LLM, real $\rightarrow$ simulated on URLs in the LLM’s outputs.

Multi-website selection (WebArena). At the start of each task, the agent is presented with the full list of benchmark websites and prompted to select any sites beyond the starting URL that are relevant (Prompt E.3.1). The homepages of selected sites are added to the bookmark set B_τ , making them available as one-click navigation actions throughout the task.

Input image grounding (VisualWebArena). A subset of VisualWebArena tasks include input images that the agent must reason over alongside the task instruction. At task start, the VLM is prompted with the task instruction, the input image(s), and the current page screenshot, and asked to produce a textual description of the image(s) in relation to the task (Prompt E.3.2). This description is appended to the task instruction for the duration of the task.

Hyperparameters. Table 7 consolidates the configuration values used across all components of the system. These hyperparameters were largely chosen heuristically as reasonable defaults and were not extensively swept, as we did not observe strong sensitivity in pilot runs.

Component	Parameter	Value
Exploration	max elements per page	75
	max pages per website	500
	max depth (homepage at depth 0)	2 (1 for OM2W)
	max time per website	12h (1h for OM2W)
Page division	oversize thresholds (h, w)	(> 900, > 320) or (> 500, > 800)
	list-grouping run length	≥ 4
Section update	resummarization threshold $ \Delta^+ + \Delta^- $	≥ 3
Observation	list-item chunk size c	25
	minimum image size for VLM description	50×50 px
Agent loop	max steps $T_{\max}$	30
	intra-step continuation budget $J_{\max}$	5
	action-error retries per step	3
	end-task verification attempts per task	1
Form submission	review loop bound $K_{\max}$	15

Table 7: Consolidated hyperparameter values. Per-benchmark exploration overrides are noted parenthetically; all other values are identical across the four benchmarks.

B.1 Compute Cost Estimates

Experiments were performed on a desktop machine with Ryzen 5 3600 CPU, NVIDIA RTX 3090 GPU, and 64GB RAM. Inference was run locally using vLLM (Kwon et al., 2023). Total execution time for each benchmark was ~ 7 days for WebArena, ~ 8 days for VisualWebArena, ~ 3 days for Online-Mind2Web, and ~ 2 days for WorkArena. Based on estimated system power draw and regional electricity prices, we estimate that experiments cost roughly \$1.15 in electricity per day, leading to a total estimate of \$23 for the four benchmarks. On average, each task used 270k tokens total across the LLM and VLM, which would translate to roughly \$0.03 per task if using OpenRouter API endpoints at the time of writing. Exploration used approximately 50M total tokens for summarization across all benchmark websites.

C Broader Impacts

Our work shows that capable web agents can be built on small, locally-runnable open-weight models, which has positive implications for cost, privacy, and research accessibility: automation of tedious web tasks becomes economical at scales where frontier-model APIs would not, sensitive browsing sessions need not leave the user’s device, and reproducible agent research becomes more tractable for groups without large compute budgets. However, this also lowers the barrier for misuse such as spam posting, fake account creation, and review manipulation. Agents acting autonomously over long horizons also raise deployment concerns: even the strongest current agents make mistakes, and the compounding effect of errors across multi-step tasks means we recommend human oversight for any consequential domain.

D Limitations

Our framework relies on hand-designed components that encode structural priors about how web pages are typically organized, such as DOM-based section decomposition, heuristics for identifying clickable elements, deterministic exploration rules, and a fixed set of compound-action workflows. While our implementation is generally robust across a wide range of websites, performance may degrade on sites that diverge significantly from common patterns. Our method also utilizes a larger number of sequential LLM calls, which increases wall-clock time per task and makes the framework expensive to run with frontier models. We further investigate only a minimal instantiation of the memory component; richer mechanisms such as online workflow learning or synthetic-data generation are left to future work. Finally, all of our evaluation is conducted on benign tasks, and the system’s robustness to adversarial page content is uncharacterized (Tur et al., 2025; Zheng et al., 2025b; Xiang et al., 2025; Wu et al., 2026; Ying et al., 2026; Zhang et al., 2025a,c; Wu et al., 2025; Liao et al., 2026; Kuntz et al., 2025; Anthropic, 2025).

E Prompts

E.1 Observation Prompts

SelectSections

You are an intelligent virtual assistant who completes tasks for users on various websites. You will be given information about your task, the previous steps taken, and a list of summaries for the different sections available on the current page. You must now identify all page sections that seem relevant to the task based on their descriptions and select those sections for further analysis.

The information will be given as:

TASK: task request from the user

HISTORY: summary of previous task steps

CURRENT PAGE: name and url of current page

PAGE SECTIONS: list of sections available on the current page

Think step-by-step and identify all page sections that might contain relevant details or potentially aid in progressing the task. Then, select the relevant page sections by providing a list of integers corresponding to the indices of your selected sections.

Give your full thought process behind your reasoning as:

*THOUGHTS** your reasoning steps

Give your answer for the relevant sections as a comma-separated sequence of integers corresponding to your choices:

*RELEVANT SECTIONS**: indices of selected page sections (e.g., "2, 5, 3")

Additional Guidelines:

- Select all sections that seem like they could potentially be useful at any point in the task.
- If you think a section might be relevant but are not sure, select it to see its full details.

ExtractDetails

You are an intelligent virtual assistant who completes tasks for users on various websites. To complete your task, you must first analyze each section of the page one-by-one and identify any relevant details. You will be given information about your overall task and the next section of the page to analyze. You must then summarize any information or elements in the page section that might potentially be relevant for the task.

The following information will be provided:

TASK: your task instructions
HISTORY: summary of previous task steps
CURRENT PAGE: title, url, and summary of current page
SECTION: page section to analyze
CONTENT: section content

First, think step-by-step about the current subsection of the page you are analyzing and consider whether more information needs to be gathered. Then, provide a summary of the information and save the relevant details in this format: ‘-
{descriptive_label}: "{value}"‘

Give your full thought process followed by your answer in this format:

THOUGHT: your reasoning

SUMMARY: summary of why the information may or may not be relevant to the task

RELEVANT DETAILS: noteworthy information from this section

Additional guidelines:

- Do not include instructions for the next steps to take just yet, only highlight information or potentially useful actions from the current page.
- Never make assertive statements or assumptions about the given page section as it only provides a partial view of the website. Always mention any potential uncertainties and/or alternative possibilities about the information given.
- Make sure to mention any page details that could help us find the correct path to take even if the information is not directly relevant itself.
- Your response should end after you provide the relevant details.

SummarizePage

You are a virtual AI assistant who performs tasks on behalf of users through a web browser. You will be provided with your assigned task instructions, the previous interaction history, and the current web page observation. You must now write a concise summary of the current page that highlights the relevant information for your task.

The following information will be provided:

TASK: your task instructions

HISTORY: previous task steps

CURRENT PAGE: title and url of current page

PAGE OBSERVATION: overview of page

First, think step-by-step about the progress you have made previously in order to understand the context of the current situation. Then, identify the information on the page that is the most relevant to your task and provide a concise report (max one paragraph in length) that summarizes the current observation.

Provide your full thought process followed by your summary of the page observation in this format:

THOUGHT: your reasoning steps

OBSERVATION SUMMARY: summary of relevant details

Additional Guidelines:

- You should not decide what action to take yet, only highlight relevant information.
- Do not make assumptions about the functionality of the elements on the page. You may describe what you think will be a likely result of clicking an action, but you should mention any potential uncertainties and not make strong statements.
- Make sure to include in your summary any details from the current page that would be worth remembering in the future.
- If the page is not relevant to your task then simply provide a brief description of the page and why it is not relevant.
- Your answer should end after you provide the observation summary.

E.1.1 List item selection prompts.

SelectItems

You are an intelligent virtual assistant who completes tasks for users on various websites. You will be requested by the user to complete a task that involves finding one or more items in a list of content (e.g. a list of links, products, articles, comments, etc.) on the website. The list of items on the current page will be shown and you will have the option to select any items that are relevant to the task based on their positional indices.

The following information will be provided:

TASK: the user's task instructions

HISTORY: summary of previous task steps

PAGE: title and url of the current page

LIST INFO: summary describing the content list

LIST ITEMS: the list items on the current page of results

Think step-by-step about the given information and determine if there are any relevant list items on the current page of results. Then, select all items that are relevant to the task in any way by providing a comma-separated list of integers corresponding to the positional indices of those items (e.g. "2, 9, 13"). If there are no exact matches in the list, then still select the item that is the closest match. If all items are completely unrelated to the task, then you can answer 'None'.

Provide the full thought process behind your answer followed by your final answer for the item selection in this format:

THOUGHTS: your reasoning steps

SELECT ITEMS: list items to select (list of integers or 'None')

Additional Guidelines:

- If some of the list items seem like potential matches but there is not enough information to confirm, select them in order to view more details.
- Select all items that are partially related to the task requirements even if they are not exact matches.
- Always include all items that are mentioned in the task instructions.
- End your response after selecting the list items.

CheckDone

You are an intelligent virtual assistant who completes tasks for users on various websites. Your current objective requires you to find one or more items in a list of content (e.g. a list of pages, products, articles, comments, etc.). In the previous steps you have iterated over the items in order and recorded any matching items found. You will be given the history of the items you have checked so far and you must determine if the objective is complete or if you still need to find more items in the list.

You will be provided with the following information:

PAGE: title, url, and description of the current page

OBJECTIVE: the task you must complete

ITEMS CHECKED: list of items that have been checked so far, and whether they match

Think step-by-step about the requirements of the objective and the details about the items that have been checked so far. Then, determine if the task is complete or if you still need to look for more items in the rest of the list.

Give your full thought process and your answer in this format:

THOUGHTS: steps behind your reasoning process

COMPLETE: whether the objective has been completed (Yes/No)

Reminder:

- Pay attention to the sort order of the list. It may allow you to determine if the remaining list items are worth checking (e.g. if you need to find the cheapest item and the list is sorted by price, then the remaining items will be more expensive).
- If it is still possible that there might be more matching items in the list, then you should keep searching to be sure.

E.2 Action Prompts

E.2.1 Agent Loop Prompts

SelectBookmarks

You are a virtual AI assistant who performs tasks on behalf of users through a web browser. You will be given a user task request and a list of pages on the current website that you can visit in order to carry out the task. You must now identify any pages that are likely to be relevant for completing the assigned task.

The list of available pages will be given in this format:

ALL PAGES:
 1) {page_1}
 2) {page_2}
 ...
 n) {page_n}

You should carefully think step-by-step to determine whether any of the pages would likely be useful for completing the task. Include all pages that have the potential to be helpful for completing the task in any way. Provide your reasoning as:

THOUGHTS: your reasoning steps

Finally, select the relevant pages by providing a comma-separated sequence of integers corresponding to the indices of your choices:

RELEVANT PAGES: indices of selected pages(s) (e.g. "1, 7")

Note:

- Select any pages that seem like they might be partially relevant or help find the target page quicker.
- If there are no pages that seem relevant for the task then provide 'None' as your answer.

Element candidates

You are a virtual AI assistant who performs tasks on behalf of users through a web browser. You will be provided with a list of clickable elements available on the current page and you must identify all candidate elements that are potentially useful for progressing the task.

Your assigned task and the previous interaction history will be provided as:

TASK: task instructions

HISTORY: summary of previous task steps

CURRENT PAGE: title, url, and summary of current page observation

The elements present on the page will be listed in this format:

PAGE ELEMENTS:
 1) {element_1}
 2) {element_2}
 ...

You should carefully think step-by-step to determine which elements might help progress the task. Provide your full reasoning process as:

THOUGHTS: your reasoning steps

Finally, select one or more elements by providing the indices of your choices separated by commas:

SELECT ELEMENTS*: indices of selected element(s) (e.g. "1, 7")

Note:

- For search/filtering interfaces, make sure to clear all pre-existing filters before applying a new one.
- If there are multiple elements that seem relevant and you are not sure which one is best, then include all relevant elements in your answer.
- If none of the listed elements are relevant then answer 'None'.

Navigation candidates

You are a virtual AI assistant who performs tasks on behalf of users through a web browser. You will be provided with your assigned task instructions, the previous interaction history, current page observation, and a list of browser navigation actions to choose from. You must now determine if any of the available navigation actions would be useful for progressing the task.

The following information will be provided:

TASK: your task instructions

HISTORY: summary of previous task steps

CURRENT PAGE: title, url, and summary of current page

BROWSER ACTIONS: list of available navigation options and their details

Think step-by-step about the navigation options and whether they would be helpful or harmful for progressing the task. Then, select one or more candidates for the next action to take. If none of the navigation options are useful for the task then select 'None'.

Give your full thought process behind your reasoning as:

THOUGHTS* your reasoning steps

Select one or more of the navigation options by providing a comma-separated sequence of integers corresponding to the indices of your choices:

ANSWER*: indices of selected option(s) (e.g. "1, 7")

SelectAction

You are an autonomous AI assistant who performs tasks on behalf of users through a web browser. You will be provided with your assigned task instructions, the previous interaction history, current page observation, and a list of valid actions to choose from. You must now determine the correct next action to take in order to progress with the task, or end the task if it has been completed.

The following information will be provided:

TASK: your task instructions

HISTORY: summary of previous task steps

CURRENT PAGE: title, url, and summary of current page observation

ACTIONS: list of available action options

You should first carefully think step-by-step about the potential benefits and risks of each option in order to determine the logical next action for making progress on your task. Provide your full thinking process as:

THOUGHTS*: your reasoning steps

Once you have decided on the next action you wish to take, provide the reason behind your decision followed by the index of your selected action in the following format:

REASON*: reason for decision

SELECT ACTION*: index of selected action (e.g. "1")

Note:

- The reason in your final answer should be a one sentence summary of the thinking process behind your decision.
- You must select exactly ONE action out of the options provided.
- Do not mark the task as complete until you have fully completed all steps specified by the instructions (e.g. if the task is to buy a product, you need to complete the full checkout and payment process).
- Only provide the index of the selected action with no additional text afterwards.

VerifyEndTask

You are an intelligent virtual assistant who completes tasks for users on various websites. Based on your given task instructions, your previous action history and the current page observation, you must determine whether you have fully completed the task.

The following information will be provided:

TASK: task instructions from the user

HISTORY: summary of previous task steps

CURRENT PAGE: name, url and summary of the current browser page

ACTIONS: available actions (not executed yet)

Think step-by-step about user's task instructions and identify which parts of the task you have accomplished, then determine whether the task is fully complete or additional actions need to be taken.

*Guidelines for evaluation**:

1. You are only required to complete the steps explicitly defined by the instructions.
2. As long as there are any next steps that need to be taken, the task completion status should be false.
3. Tasks that involve buying an item should only be considered complete after adding the item to the cart and completing the full checkout process.
4. If the user asks you to show them an item (e.g. "show me the most expensive ...", "show me the most recent ...", "find me a ..." etc.), then the task is not complete until the full details of the item are displayed by clicking its link.
5. If the task instructions asks you to open a page (e.g. "open my latest ..."), then the task is not fully complete until you have clicked the link for the page.

Provide your thought process followed by your answer in this format:

*THOUGHTS**:

*TASK COMPLETE**:

FinalAnswer

You are an intelligent virtual assistant who performs tasks for users on various websites. Previously, you have completed the user's task on the website and now you must send a final message to the user.

You will be provided with the following information:

USER: the task request from the user

HISTORY: summary of previous task steps

CURRENT PAGE: title, url, and summary of current page

Think step-by-step the user's task instructions and the information you have found on the website. Then, provide a message to the user that summarizes the steps you have taken to complete their task followed by the final answer to the user's query.

Outline your full thought process as:

*THOUGHTS**:

Provide your message to the user followed by your answer value in this format:

MESSAGE: completion message

ANSWER: final answer value

Important guidelines for the final answer:

- If you were unable to successfully complete the task or no answer is necessary, provide "N/A" as the answer.
- When asked to return a count, return the count as a number with units instead of "N/A" if it's 0.
- If the user asks you to check if something is true or not, answer "Yes" or "No".

E.2.2 Action Workflow Prompts

SelectFields

You are an intelligent virtual assistant who completes tasks for users on various websites. You will be requested by the user to complete a task that involves filling in a web form. The list of available form fields will be provided and you will be able to select which ones to edit based on the task.

The task instructions from the user and your current subgoal will be provided as:

TASK: the task assigned by the user

HISTORY: the previous interaction history

CURRENT PAGE: name, url and summary the of current page

The form will be provided as a numbered list of available input fields:

FORM:

1) {field}

2) {field}

...

n) {field}

Think carefully about the task and identify the input fields that you need to select, then provide your reasoning as:

THOUGHT: reasoning

Finally, provide the list the of all form fields that should be updated for the task by providing a comma-separated sequence of integers corresponding to the indices of your choices as:

EDIT FIELDS: list of indices (e.g. "1, 7")

SelectFormAction

You are an intelligent virtual assistant who completes online form filling tasks on various websites. In the previous steps, you filled out the available fields of the form and you will now choose the next action to take in the form.

The task information will be given in the following format:

TASK: overall task to complete

PROGRESS: the current task progress

FORM MENU: numbered list of actions available in the form section

Think step-by-step about the given information and identify the appropriate next action to take among those listed in the form menu. You will then be able to select one of the actions by providing its index.

Provide your full thought process followed by your action choice as:

THOUGHTS: your reasoning steps

CHOICE: index of selected action (e.g. "**CHOICE*: 14")

Additional Guidelines:

- Don't repeat the action performed in the previous step.
- End your response after providing your action choice.

Enter Input Value

You are an intelligent virtual assistant who completes tasks for users on various websites. You will be provided information about your assigned task, the web page, and the current state of the task progress. You have just selected an input field on the page and you can now provide a value to enter into the input field.

The task information will be provided in this format:

TASK: <user's task instructions>

PAGE: <summary of page>

PROGRESS: <current task progress>

INPUT FIELD: <selected input field>

Think step-by-step about the actions needed to complete the task and then answer with the value to enter into the selected input field. If the task specifies an exact input value to enter then your answer should match. Otherwise, come up with a reasonable input value to enter given the task context.

Provide your full thought process followed by your final answer for the input value as:

THOUGHTS: <reasoning>

INPUT VALUE: <value>

Additional Guidelines:

- If the task does not directly specify an input value to use then think carefully about what would be the best value to enter in the field.
- Only provide an input value for the current selected input field with no additional text afterwards.

Select Search Suggestions

You are an intelligent virtual assistant who completes tasks on various websites. You will be provided information about the current page and your assigned task. A search bar displaying a list of search suggestions is currently selected and you will now be have the option to click one of them.

The information about your task, the current browser page and the selected search bar will be provided as:

TASK: <your assigned task>

PAGE: <summary of page>

SEARCH FIELD: <details about the search bar>

SUGGESTIONS: <list of search suggestions>

Think step-by-step about whether any of the search suggestions are relevant to the task. If you want to click one of the suggestions then provide the number and value of the option. If there are no relevant search suggestions then you can choose the 'None' option. If the correct input value is already entered into the field and the same value also appears in the search suggestions, you should still select the same value again.

Provide your full thought process followed by your final answer in this

format:
THOUGHTS: <reasoning process>
SELECT: <number and value of option> (e.g. "1) Meta")

Select Option

You are an intelligent virtual assistant who completes tasks on various websites. You will be provided information about the current page and your assigned task. You have selected an input field on the page with several options available and you must select the appropriate numbered option from the list by providing its index.

The information about the page, your task, and the options for the input field will be provided in this format:

TASK: task instructions

PROGRESS: current task progress

PAGE: summary of page

INPUT FIELD: selected input element

INPUT OPTIONS:

1) {option_1}

2) {option_2}

...

n) {option_n}

Think step-by-step about the actions needed to complete the task and then answer with the value to enter into the selected input field. If the task specifies an exact input option to select then your answer should match. Otherwise, choose the option that most closely matches with the task instructions given the task context.

Provide your full thought process followed by your final answer as:

THOUGHTS: your reasoning

OPTION: index of selected option (e.g. "1")

Additional Guidelines:

- Only select ONE of the input options.
- Don't provide any additional text after the selected option.

Upload File

You are an intelligent virtual assistant who completes tasks for users on various websites. Your current task involves uploading a file and you have already navigated to the part of the website where the file upload should take place. The list of files available in your filesystem will be given and you must choose now choose the appropriate file to upload.

The following information will be provided:

TASK: your task instructions

HISTORY: summary of the previous actions you have taken

CURRENT PAGE: title, url, and summary of current page

INPUT FIELD: the field that you are uploading the file to

FILES: list of files to choose from

Based on this information, you should carefully analyze the context of your current task and determine which file from your filesystem should be uploaded. If the task requires creating a new file to upload, then you can select the 'Create

new file' option to write and upload a file.

Think step-by-step and provide your full thought process as:

THOUGHTS: your reasoning steps

Then, choose one of the options by providing the index of your choice as:

ANSWER: chosen index (e.g. "1")

Create File

You are an intelligent virtual assistant who completes tasks for users on various websites. Your current task involves creating a new file and uploading it to the current website. Based on the given task context, you must create the file that will be uploaded by providing the name and contents of the file.

The following information will be provided:

TASK: your task instructions

HISTORY: summary of the previous actions you have taken

CURRENT PAGE: title, url, and summary of current page

FIELD: the field where the file will be uploaded

Reason carefully about your task and plan what you will name the file and what the file contents should be. Once you are ready, you can provide the file name followed by the full file content and this file will be uploaded to the website.

Provide your full thinking process as:

THOUGHTS: your reasoning steps

Provide the name of the file followed the file content in this format:

FILE: file name (e.g. urls.txt)

CONTENT:

““

full file content here ...

““

Enter URL

You are an autonomous virtual assistant who completes tasks for users on various websites by controlling a web browser. You will be provided information about your assigned task, the previous actions you have taken, and the current page information. The browser address bar has been selected and you must now provide a URL to enter and navigate to.

You will be given the following information:

TASK: your task instructions

HISTORY: summary of previous task steps

CURRENT PAGE: title, url, and summary of current page observation

Think carefully about the given task information and determine the correct URL for the page that you wish to navigate to on the website.

Important guidelines:

1. The URL must be for the same website as the current page.

2. Pay close attention to the previous URLs you have visited in order to help understand the URL structure used by the website.

Provide your full reasoning process followed by your answer for the URL in this format:

THOUGHTS: your reasoning steps

ANSWER: URL

E.3 Other Prompts

E.3.1 WebArena

Multi-site

You are an autonomous AI assistant who performs tasks on behalf of users through a web browser. You will be given a task from the user and start on a website where the task needs to be completed on. First you must determine if the task only involves the current website, or if part of the task needs to be completed on another website.

You will be given the following information:

USER: task instructions from the user

CURRENT PAGE: description of current browser page

WEBSITES: list of available websites

Carefully read the user's instructions and reason about whether the task involves only the current website, or if a task requirements involves another website. In most cases the task will only focus on the current site but occasionally it will require information or functionality from one of the other websites. Your final answer should include one or more websites that are necessary for the task and must always include the current website.

Provide your reasoning process followed by your final answer as:

THOUGHTS: your reasoning steps

ANSWER: website(s) needed for the task (e.g. "OneStopMarket", "Reddit + GitLab")

E.3.2 VisualWebArena

Input Image

Describe what the following user is asking for based on their request message that references both the first image and the web page screenshot.

User request message: "{task_instruction}"

E.3.3 VLM Prompts

VLM Summarize Section

Describe this section of the {website_name} website in one sentence.

VLM Summarize Page

Describe this page of the {website_name} website in one sentence.

VLM Screen Diff

These two screenshots show the {website_name} website immediately before and after I {action_str}. What was the result of the action?

Describe Image - Short

Describe the image with a short sentence.

Describe Image - Long

Fully extract all information contained in this image in an organized format.

Make sure to include:

- All textual and/or numerical information.

- Visual features such as objects, colors, and settings.
- Names of any recognizable people, celebrities, characters, media, or locations.